Задачи с собеседований по Си

Junior · Middle · Senior — с полным разбором решений

Учебник «Алгоритмы и игры на Си»

Содержание

Как готовиться к собеседованию по Си **1**
Что проверяют на каждом уровне . . . 1
Как пользоваться задачами . . . 2
Частые ошибки на собеседовании (и как их избежать) . . . 2
Что повторить перед собеседованием . . . 2

Приложение А. Задачи с собеседований: Junior **4**
Как пользоваться . . . 4

Часть 1. Задачи на код **6**
Задача 1. Поменять местами две переменные . . . 6
Задача 2. Проверить чётность числа . . . 7
Задача 3. Максимум и минимум массива . . . 8
Задача 4. Развернуть строку на месте . . . 9
Задача 5. Проверка палиндрома . . . 11
Задача 6. Сколько цифр в числе . . . 12
Задача 7. Сумма цифр числа . . . 13
Задача 8. Подсчёт гласных в строке . . . 14
Задача 9. Факториал итеративно . . . 15
Задача 10. FizzBuzz . . . 17
Задача 11. Второй по величине элемент массива . . . 18
Задача 12. Проверка на простое число . . . 19

Часть 2. Каверзы: «что выведет / в чём ошибка» **21**
Каверза 1. `sizeof('a')` — сколько это? . . . 21
Каверза 2. Целочисленное деление `5 / 2` . . . 21
Каверза 3. `printf("%d", c)` для `char` . . . 22
Каверза 4. `=` вместо `==` в условии . . . 22
Каверза 5. Лишняя `;` после `for(...)` . . . 23
Каверза 6. `printf("%d %d", i++, ++i)` . . . 23
Каверза 7. Неинициализированная переменная . . . 24
Каверза 8. Остаток от отрицательного числа . . . 24

Мини-резюме **25**

Куда дальше **26**

Приложение Б. Задачи с собеседований: Middle **27**
Как пользоваться этим приложением . . . 27

Часть 1. Задачи на код . . . 28
Задача 1. Своя `strlen`, `strcpy`, `strcmp` . . . 28
Задача 2. Развернуть односвязный список . . . 30
Задача 3. Середина списка (быстрый и медленный указатель) . . . 31
Задача 4. Цикл в списке (черепаха и заяц, алгоритм Флойда) . . . 32
Задача 5. Подсчёт единичных битов (popcount) . . . 33
Задача 6. Степень ли двойки . . . 35
Задача 7. Удалить дубликаты из отсортированного массива на месте . . . 36
Задача 8. Число, встречающееся нечётное число раз (XOR) . . . 37
Задача 9. Корректный бинарный поиск . . . 38
Задача 10. Слияние двух отсортированных массивов . . . 39
Задача 11. Поменять местами два числа через указатели . . . 40
Часть 2. Каверзы . . . 41
Каверза 1. `char *s = "abc"` против `char s[] = "abc"` . . . 42
Каверза 2. `sizeof` массива внутри функции . . . 42
Каверза 3. `arr[1]`, `1[arr]` и `*(arr + 1)` . . . 43
Каверза 4. `++*p`, `*p++`, `*++p`, `(*p)++` . . . 43
Каверза 5. `const char *p` против `char * const p` . . . 44
Каверза 6. Найди баг: утечка памяти . . . 44
Каверза 7. Найди баг: use-after-free и двойной free . . . 45
Каверза 8. Возврат адреса локальной переменной . . . 46
Мини-резюме . . . 46
Что дальше . . . 47

Приложение B. Задачи с собеседований: Senior **48**

Часть 1. Задачи на код **49**
Задача 1. Быстрая сортировка (quicksort) . . . 49
Задача 2. Сортировка слиянием (merge sort) . . . 51
Задача 3. Обобщённый стек через `void *` . . . 53
Задача 4. Свои `atoi` и `itoa` с обработкой краёв . . . 55
Задача 5. Таблица обработчиков на указателях на функции . . . 57
Задача 6. Кольцевой буфер (ring buffer) . . . 59
Задача 7. Хеш-таблица «строка → счётчик» с цепочками . . . 61
Задача 8. Точка пересечения двух односвязных списков . . . 63
Задача 9. Набросок LRU-кэша . . . 64
Задача 10. Разворот числа с защитой от переполнения . . . 67

Часть 2. Каверзы и теория **69**
Каверза 1. Что такое UB и где оно прячется . . . 69
Каверза 2. Strict aliasing — почему `int *` и `float *` нельзя путать . . . 70
Каверза 3. `volatile` — зачем он нужен . . . 71
Каверза 4. Две роли `static` . . . 71
Каверза 5. Stack vs heap и причины segfault . . . 72
Каверза 6. Выравнивание и padding: что выведет `sizeof` . . . 72
Каверза 7. Опасный макрос `#define SQR(x) x*x` . . . 73
Каверза 8. Идиома `do { ... } while(0)` в макросах . . . 74
Каверза 9. Порядок вычисления аргументов не определён . . . 74

Каверза 10. Endianness — порядок байтов 75

Итог практикума **76**
Напутствие 76

Как готовиться к собеседованию по Си

Следующие три приложения — это задачи с собеседований на C-разработчика, разбитые по уровням: Junior, Middle, Senior. Прежде чем браться за них, полезно понять, **что вообще проверяют** на таких собеседованиях и как готовиться эффективно. Этот короткий раздел — карта местности.

Что проверяют на каждом уровне

Уровни условны (в разных компаниях планка своя), но общая логика такая.

Junior. Тебя проверяют на «ты вообще умеешь писать код». Базовый синтаксис, циклы, массивы, функции, простые алгоритмы (развернуть строку, найти максимум, проверить палиндром). Любят каверзные вопросы на понимание основ: чему равно `5 / 2`, что напечатает `sizeof('a')`, в чём разница `=` и `==`. Здесь не ждут блестящих алгоритмов — ждут аккуратности и понимания того, что ты пишешь.

Middle. Главная тема — **указатели и память**. Своя `strlen`, операции со связными списками, работа с битами, понимание, чем `char *s = "abc"` отличается от `char s[] = "abc"`, что такое «распад» массива в указатель, где живут утечки и use-after-free. Алгоритмы уже посерьёзнее: бинарный поиск без ошибок на единицу, цикл Флойда, слияние массивов. Проверяют, что ты понимаешь, **что происходит в памяти**, а не просто заучил рецепты.

Senior. Тонкости языка и инженерное мышление. Неопределённое поведение (UB) и его источники, `volatile`, две роли `static`, выравнивание и padding в структурах, опасности макросов, порядок вычислений, endianness. Из алгоритмов — быстрая сортировка и сортировка слиянием, обобщённые структуры через `void *`, хеш-таблицы, кэш LRU. Здесь важно не только решить, но и обсудить компромиссы: сложность, память, краевые случаи, что сломается в продакшене.

Как пользоваться задачами

1. **Сначала реши сам.** Прочитай условие, закрой решение, возьми бумагу или редактор. Даже неудачная попытка включает голову так, как чтение готового ответа никогда не включит.
2. **Проговаривай вслух.** На собеседовании ценят не только ответ, но и ход мысли. Тренируйся объяснять, что и почему ты делаешь.
3. **Скомпилируй и запусти.** У каждой задачи на код есть файл в `code/`. Собери его с `-Wall -Wextra`, поиграй с входными данными, сломай нарочно — так понимание закрепляется.
4. **Разбери краевые случаи.** Пустой ввод, ноль, отрицательные числа, массив из одного элемента, переполнение. Интервьюер почти всегда спросит «а что, если…».

Частые ошибки на собеседовании (и как их избежать)

- **Молчать.** Тишина пугает интервьюера сильнее неверного ответа. Думай вслух: «так, мне нужно… я попробую вот так… здесь может быть проблема с границей».
- **Бросаться в код сразу.** Сначала уточни условие и проговори план. «Правильно ли я понял: массив отсортирован? могут быть дубликаты?»
- **Забывать про краевые случаи.** Пустой массив и переполнение `int` — любимые ловушки. Проверяй их сам, не дожидаясь вопроса.
- **Путать `=` и `==`, забывать `&` в `scanf`, выходить за границы массива.** Базовые ошибки на собеседовании читаются как «невнимательный».
- **Игнорировать память.** Выделил через `malloc` — скажи, где сделаешь `free`. Это сигнал, что ты понимаешь Си, а не просто пишешь на нём.
- **Спорить, когда не прав.** Если интервьюер показал ошибку — признай, разберись, исправь. Умение принимать обратную связь ценят высоко.

Что повторить перед собеседованием

- **Указатели**: `&`, `*`, указатель на указатель, указатель и массив, передача в функцию по адресу (глава 3).
- **Память**: stack против heap, `malloc`/`free`, утечки, use-after-free, double free (главы 3 и 11).
- **Строки**: устройство через `'\0'`, почему `==` не сравнивает строки (главы 3 и 10).
- **Биты**: `&`, `|`, `^`, `<<`, `>>`, приёмы вроде `n & (n - 1)`.
- **Сложность**: O(n), O(n log n), O(n^2) — уметь оценить свой код.
- **UB**: знать, что переполнение знакового `int`, выход за границы и запись в строковый литерал — неопределённое поведение.

Не пытайся выучить всё за ночь. Пройди задачи по уровням, начиная с Junior, даже если метишь выше: простые задачи быстро освежают базу, а каверзы из них всплывают на собеседованиях любого уровня. Поехали.

Приложение А. Задачи с собеседований: Junior

Книга закончилась, а собеседования только начинаются. Это приложение — сборник задач, которые реально дают на собеседовании С-разработчику уровня **Junior** (младший, начинающий). Здесь нет ничего, чего ты не проходил: всё опирается на главы 1–18 — числа, массивы, строки, указатели, циклы, функции. Новой теории не будет. Будет другое — взгляд на знакомые приёмы глазами интервьюера.

Зачем это нужно. На собесе мало написать работающий код. Спрашивающий смотрит, **как ты думаешь**: учёл ли краевые случаи, помнишь ли про переполнение, понимаешь ли, что делает каждая строка. Поэтому к каждой задаче я добавил блок «что спросят дальше» — те самые уточняющие вопросы, которыми тебя будут «дожимать».

Как пользоваться

Каждая задача оформлена так:

- **Условие** — как её формулируют вслух на собесе.
- **Идея** — как к ней подступиться, без кода.
- **Решение** — рабочий код.
- **Разбор** — почему так, а не иначе.
- **Что спросят дальше** — продолжения и подвохи.

Главный совет: **сначала реши сам**, на бумаге или в редакторе, и только потом смотри решение. Подсмотренное решение забывается к утру. Решённое самому — остаётся. Даже если решил криво — сравни с моим вариантом, и разница тебе многое объяснит.

Раздел делится на две части. Сначала **задачи на код** — где надо что-то написать. Потом блок **каверз** «что выведет / в чём ошибка» — короткие фрагменты, на которых проверяют, насколько ты внимателен к деталям языка.

Рабочие решения лежат в `code/19-XX-*.c` — их можно собрать и запустить. Каверзы намеренно содержат ошибки или неопределённое поведение, поэтому в файлы они не вынесены: разбираем их прямо здесь.

Часть 1. Задачи на код

Задача 1. Поменять местами две переменные

Условие. Есть две переменные `a` и `b`. Поменяй их значения местами. После обмена в `a` должно лежать то, что было в `b`, и наоборот.

Идея. Наивная попытка «в лоб» не работает:

```
a = b;   /* старое значение a потеряно навсегда */
b = a;   /* сюда попадёт b, а не старое a */
```

Первая же строка затирает `a`. Нужна **временная переменная**, чтобы запомнить значение перед затиранием.

Решение (`code/19-01-swap.c`):

```c
#include <stdio.h>

int main(void)
{
    int a = 5;
    int b = 9;

    printf("До:    a = %d, b = %d\n", a, b);

    int temp = a;   /* запомнили a */
    a = b;          /* в a положили b */
    b = temp;       /* в b положили старое a */

    printf("После: a = %d, b = %d\n", a, b);
    return 0;
}
```

Запуск:

```
До:    a = 5, b = 9
После: a = 9, b = 5
```

Разбор. `temp` хранит копию `a`, пока мы перезаписываем `a` значением `b`. Потом достаём из `temp` старое `a` и кладём в `b`. Три строки, всё прозрачно.

Что спросят дальше. «А можно без временной переменной?» Да, два способа.

Через арифметику:

```c
a = a + b;
b = a - b;   /* = старое a */
a = a - b;   /* = старое b */
```

Через XOR (исключающее «или», оператор `^`):

```c
a = a ^ b;
b = a ^ b;
a = a ^ b;
```

Оба выглядят эффектно, и интервьюер любит про них спросить. Но честный ответ — **в реальном коде так делать не надо**. Почему:

- Арифметический способ **переполняется**: если `a + b` вылезет за пределы `int`, получишь мусор (см. главу 4 про переполнение).
- XOR-своп ломается, если `a` и `b` — **одна и та же переменная** (один адрес): `x ^ x` даёт 0, и значение теряется. Для `swap(&x, &x)` это реальная ловушка.
- Оба варианта **нечитаемы** и не быстрее: современный компилятор и так оптимально реализует обмен через `temp`.

Правильный вывод, который хочет услышать грамотный интервьюер: «знаю про XOR-трюк, но в продакшене пишу через `temp` — понятнее и без подвохов».

Задача 2. Проверить чётность числа

Условие. Дано целое число. Определи, чётное оно или нечётное.

Идея. Школьный способ — остаток от деления: `n % 2 == 0` (см. главу 4). Но на собесе часто хотят увидеть **битовый** приём: у чётного числа младший бит равен 0, у нечётного — 1. Проверить младший бит можно операцией `&` (побитовое «И») с единицей.

Решение:

```c
#include <stdio.h>

int main(void)
{
    int n = 7;

    if (n & 1) {
        printf("%d — нечётное\n", n);
    } else {
        printf("%d — чётное\n", n);
    }
    return 0;
}
```

Разбор. `n & 1` сравнивает биты `n` и числа 1 по позициям. У единицы выставлен только самый младший бит. Поэтому `n & 1` оставляет от `n` ровно этот младший бит: 1 у нечётных, 0 у чётных. В `if` ненулевое — это истина (см. главу 2), так что `if (n & 1)` срабатывает на нечётных.

Что спросят дальше. «Чем `n & 1` лучше `n % 2`?» Для положительных чисел — ничем существенным, оба корректны и одинаково быстры. Но есть тонкость с **отрицательными**: в C `-7 % 2` даёт `-1` (мы это проверим в каверзах), поэтому проверка `n % 2 == 1` для отрицательных нечётных **не сработает** — там получится `-1`, а не `1`. Безопаснее писать `n % 2 != 0`. А `n & 1` на типичной машине даёт правильный младший бит и для отрицательных. Это хороший момент показать, что ты знаешь про знак.

Задача 3. Максимум и минимум массива

Условие. Дан массив целых. Найди наибольший и наименьший элементы за один проход.

Идея. Заводим `max` и `min`, инициализируем их **первым элементом** массива (не нулём! см. подвох ниже) и идём по остальным, обновляя.

Решение:

```c
#include <stdio.h>

int main(void)
{
    int a[] = {4, -2, 9, 1, 7, -8, 3};
    int n = (int)(sizeof(a) / sizeof(a[0]));
```

```
    int max = a[0];
    int min = a[0];
    for (int i = 1; i < n; i++) {
        if (a[i] > max) {
            max = a[i];
        }
        if (a[i] < min) {
            min = a[i];
        }
    }

    printf("min = %d, max = %d\n", min, max);
    return 0;
}
```

Разбор. `sizeof(a) / sizeof(a[0])` — это размер всего массива в байтах, делённый на размер одного элемента, то есть **число элементов**. Приём работает только пока `a` — настоящий массив, а не указатель (см. ниже). Цикл стартует с `i = 1`, потому что нулевой элемент уже лежит в `max` и `min`.

Что спросят дальше.

- «Почему инициализируешь первым элементом, а не нулём?» Если поставить `max = 0`, а все числа в массиве отрицательные — максимумом ошибочно останется 0, которого в массиве нет. То же с `min` и положительными. Безопасно стартовать значением **из самого массива**.
- «А если передать массив в функцию и там взять `sizeof`?» Не сработает! В функции параметр-массив превращается в **указатель**, и `sizeof` вернёт размер указателя (8 байт), а не массива. Поэтому длину всегда передают **отдельным параметром**: `void f(int *a, int n)`. Это классический подвох (см. главу 3 про указатели и каверзу про `sizeof` ниже).

Задача 4. Развернуть строку на месте

Условие. Дана строка. Переверни её **на месте** — без создания второго массива.

Идея. Два указателя-индекса: один на начало (`left`), другой на конец (`right`). Меняем символы местами и идём навстречу, пока они не встретятся посередине.

Решение (`code/19-02-reverse-string.c`):

```c
#include <stdio.h>
#include <string.h>

/* Переворачивает строку на месте, без второго массива. */
void reverse(char *s)
{
    int left = 0;
    int right = (int)strlen(s) - 1;
    while (left < right) {
        char tmp = s[left];
        s[left] = s[right];
        s[right] = tmp;
        left++;
        right--;
    }
}

int main(void)
{
    char text[] = "Hello, world";
    reverse(text);
    printf("%s\n", text);
    return 0;
}
```

Запуск:

```
dlrow ,olleH
```

Разбор. `strlen(s)` — длина строки без учёта завершающего нуля `'\0'` (см. главу 10). Последний символ имеет индекс `strlen(s) - 1`. Внутри цикла — обычный обмен через `tmp` (как в задаче 1). Условие `left < right` останавливает нас на середине: средний символ при нечётной длине трогать не нужно.

Что спросят дальше.

- «Почему `char text[] = "..."`, а не `char *text = "..."`?» Строковый литерал через указатель (`char *`) — это, как правило, **read-only** память; попытка её менять — неопределённое поведение и падение. Массив `char text[]` — это твоя изменяемая копия. Менять можно только её. Это важная деталь (см. каверзу про литералы ниже).

- «Сложность?» O(n) по времени, O(1) по дополнительной памяти — мы не выделяли новый буфер, переставляли на месте.

Задача 5. Проверка палиндрома

Условие. Палиндром читается одинаково слева направо и справа налево (`level`, `radar`). Проверь, палиндром ли данная строка.

Идея. Та же пара указателей с краёв, но теперь не меняем символы, а **сравниваем**. Нашли несовпадение — сразу не палиндром.

Решение (`code/19-03-palindrome.c`):

```c
#include <stdio.h>
#include <string.h>

/* Возвращает 1, если строка — палиндром, иначе 0. */
int is_palindrome(const char *s)
{
    int left = 0;
    int right = (int)strlen(s) - 1;
    while (left < right) {
        if (s[left] != s[right]) {
            return 0;   /* нашли несовпадение — это не палиндром */
        }
        left++;
        right--;
    }
    return 1;
}

int main(void)
{
    const char *words[] = {"level", "radar", "hello", "noon"};
    for (int i = 0; i < 4; i++) {
        if (is_palindrome(words[i])) {
            printf("%s — палиндром\n", words[i]);
        } else {
            printf("%s — нет\n", words[i]);
        }
```

```
    }
    return 0;
}
```

Запуск:

```
level — палиндром
radar — палиндром
hello — нет
noon — палиндром
```

Разбор. Параметр `const char *s` — обещание «эту строку я менять не буду», только читаю. Поэтому функция спокойно принимает строковые литералы. Выход `return 0` при первом же несовпадении — это **ранний выход**: нет смысла проверять дальше, ответ уже ясен.

Что спросят дальше.

- «Сделай регистронезависимо: `Level` тоже палиндром.» Сравнивай не `s[left]` и `s[right]`, а их версии в одном регистре (приём приведения к нижнему регистру — в задаче 8 про гласные).
- «А если игнорировать пробелы и знаки: `A man a plan a canal Panama`?» Тогда указатели нужно «перепрыгивать» через не-буквы. Часто это и есть настоящая задача за вывеской простого палиндрома.

Задача 6. Сколько цифр в числе

Условие. Дано целое число. Посчитай, сколько в нём цифр (123 → 3).

Идея. Делим число на 10 нацело, пока оно не станет нулём, и считаем число шагов (см. главу 4 — «откусывание» цифр).

Решение. Подсчёт цифр объединён с суммой цифр в одном файле `code/19-08-digits.c` — см. задачу 7. Ядро подсчёта:

```
int count = 0;
if (n == 0) {
    count = 1;   /* у нуля одна цифра */
}
while (n > 0) {
    count++;
    n /= 10;     /* то же, что n = n / 10 */
}
```

Разбор. Каждое деление на 10 убирает одну цифру справа, и так до нуля. Отдельный случай — само число **0**: цикл `while (n > 0)` для него не выполнится ни разу, а ответ должен быть 1 (одна цифра). Поэтому ноль обрабатываем вручную.

Что спросят дальше. «А отрицательные?» У −123 три цифры, минус не считается. Возьми модуль в начале: `if (n < 0) n = -n;`. И сразу подвох: `-n` для самого маленького `int` (`INT_MIN`) переполнится — у него нет положительной пары. На Junior это редко спрашивают, но упомянуть — большой плюс.

Задача 7. Сумма цифр числа

Условие. Посчитай сумму цифр числа (456 → 4+5+6 = 15).

Идея. То же «откусывание», но вместо счётчика копим сумму остатков `n % 10`.

Решение (`code/19-08-digits.c` — считает и количество, и сумму):

```c
#include <stdio.h>

int main(void)
{
    int n;
    printf("Введи целое число: ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }

    if (n < 0) {
        n = -n;   /* знак на количество и сумму цифр не влияет */
    }

    int count = 0;   /* сколько цифр */
    int sum = 0;     /* сумма цифр */

    if (n == 0) {
        count = 1;   /* у нуля одна цифра */
    }
    while (n > 0) {
        int digit = n % 10;
        sum += digit;
```

```
        count++;
        n /= 10;
    }

    printf("Цифр: %d, сумма цифр: %d\n", count, sum);
    return 0;
}
```

Запуск:

```
Введи целое число: 90834
Цифр: 5, сумма цифр: 24
```

Разбор. `n % 10` — последняя цифра, `n /= 10` — отбрасываем её. `digit` вычисляем **один раз** и используем для суммы, чтобы не повторять `n % 10` (см. главу 4, шаг 3). `count` и `sum` стартуют с 0: сумма копится с нуля, как и положено накопителю.

Что спросят дальше. «Цифровой корень: складывай сумму цифр, пока не останется одна цифра (9875 → 29 → 11 → 2).» Это сумма цифр в цикле поверх цикла — частое продолжение.

Задача 8. Подсчёт гласных в строке

Условие. Посчитай, сколько гласных букв в строке (английские a, e, i, o, u в любом регистре).

Идея. Идём по строке посимвольно до `'\0'`. Для каждого символа проверяем, не гласная ли он. Чтобы не плодить проверки для верхнего и нижнего регистра, приводим символ к нижнему.

Решение (`code/19-06-vowels.c`):

```
#include <stdio.h>
#include <string.h>

/* Возвращает 1, если c — английская гласная (любого регистра). */
int is_vowel(char c)
{
    char lower = c;
    if (c >= 'A' && c <= 'Z') {
        lower = (char)(c - 'A' + 'a');   /* привели к нижнему регистру */
    }
    return lower == 'a' || lower == 'e' || lower == 'i'
        || lower == 'o' || lower == 'u';
}
```

```c
int main(void)
{
    const char *s = "Programming In C";
    int count = 0;
    for (int i = 0; s[i] != '\0'; i++) {
        if (is_vowel(s[i])) {
            count++;
        }
    }
    printf("В строке \"%s\" гласных: %d\n", s, count);
    return 0;
}
```

Запуск:

```
В строке "Programming In C" гласных: 4
```

Разбор. Трюк `c - 'A' + 'a'` переводит заглавную букву в строчную. Символы в C — это коды (см. главу 2): буквы `A..Z` идут подряд, и `a..z` тоже. Разница между заглавной и строчной одной буквы постоянна (`'a' - 'A'`), поэтому сдвиг работает. Условие цикла `s[i] != '\0'` — канонический способ пройти строку до завершающего нуля.

Что спросят дальше.

- «А `<ctype.h>`?» В стандартной библиотеке есть `tolower()` — он делает ровно наш сдвиг и работает с любой локалью. На собесе ценят, когда ты знаешь про готовое, но умеешь и руками.
- «Учитывай ‘y’ / русские гласные» — расширение списка, проверка внимательности.

Задача 9. Факториал итеративно

Условие. Посчитай факториал числа `n` циклом (без рекурсии). `n! = 1·2·…·n`, при этом `0! = 1`.

Идея. Накопитель `fact = 1`, домножаем на 2, 3, …, n. Стартовое значение — **единица**, а не ноль (умножение на 0 всё обнулит). И сразу помним про переполнение из главы 4.

Решение (`code/19-04-factorial.c`):

```c
#include <stdio.h>

int main(void)
{
```

```c
    int n;
    printf("Факториал какого числа? ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }

    if (n < 0) {
        printf("Факториал определён только для n >= 0.\n");
        return 1;
    }
    if (n > 20) {
        printf("Слишком большое: long long переполнится при n > 20.\n");
        return 1;
    }

    long long fact = 1;   /* 0! = 1 и 1! = 1 — стартуем с единицы */
    for (int i = 2; i <= n; i++) {
        fact = fact * i;
    }

    printf("%d! = %lld\n", n, fact);
    return 0;
}
```

Запуск:

```
Факториал какого числа? 13
13! = 6227020800
```

Разбор. Тип накопителя — `long long` (8 байт), потому что `int` лопается уже на 13! (см. главу 4). Печатаем через `%lld`. Цикл идёт с 2 (умножать на 1 бессмысленно); для `n = 0` и `n = 1` он не выполнится, и `fact` останется 1 — верный ответ.

Что спросят дальше.

- «До какого `n` это корректно?» До 20: 20! ≈ 2.4·10^18 ещё помещается в `long long`, а 21! — уже нет. Поэтому мы заранее отсекаем `n > 20`. Грамотный кандидат **сам** проговаривает границу типа.
- «А рекурсией?» Можно (см. главу 9), но итеративно — без риска переполнить стек вызовов на больших `n` и обычно быстрее.

Задача 10. FizzBuzz

Условие. Числа от 1 до 100. Кратные 3 заменяй на `Fizz`, кратные 5 — на `Buzz`, кратные и 3, и 5 — на `FizzBuzz`, остальные печатай как есть.

Идея. Вся соль — в **порядке** проверок. Самое строгое условие (кратно 15) должно идти первым, иначе число 15 поймается веткой «кратно 3» и выдаст `Fizz` вместо `FizzBuzz` (полный разбор — в главе 4).

Решение (готовый файл — `code/04-fizzbuzz.c` из главы 4):

```c
#include <stdio.h>

int main(void)
{
    for (int i = 1; i <= 100; i++) {
        if (i % 15 == 0) {
            printf("FizzBuzz\n");
        } else if (i % 3 == 0) {
            printf("Fizz\n");
        } else if (i % 5 == 0) {
            printf("Buzz\n");
        } else {
            printf("%d\n", i);
        }
    }
    return 0;
}
```

Разбор. В цепочке `if / else if` срабатывает **первая** подходящая ветка. Поэтому «кратно 15» (то же, что «кратно и 3, и 5») ставим раньше «кратно 3» и «кратно 5».

Что спросят дальше. «Сделай без `% 15`, только через `% 3` и `% 5`.» Собери строку из флагов:

```c
int by3 = (i % 3 == 0);
int by5 = (i % 5 == 0);
if (by3) printf("Fizz");
if (by5) printf("Buzz");
if (!by3 && !by5) printf("%d", i);
printf("\n");
```

Этот вариант легко расширяется (добавить «кратно 7 → Bazz»), и интервьюер часто просит именно его — проверить, что ты не зубрил, а понимаешь структуру.

Задача 11. Второй по величине элемент массива

Условие. Найди второй по величине элемент массива за **один** проход (не сортируя).

Идея. Держим две переменные: `max` (наибольший) и `second` (второй). Для каждого числа: если оно больше `max` — старый `max` становится `second`, а само число — новым `max`. Иначе, если оно меньше `max`, но больше `second` — обновляем `second`. Дубликаты максимума вторым считать не должны.

Решение (`code/19-05-secondmax.c`):

```c
#include <stdio.h>
#include <limits.h>

int main(void)
{
    int a[] = {3, 9, 9, 1, 7, 9, 4};
    int n = (int)(sizeof(a) / sizeof(a[0]));

    int max = INT_MIN;     /* самое большое */
    int second = INT_MIN;  /* второе по величине */

    for (int i = 0; i < n; i++) {
        if (a[i] > max) {
            second = max;  /* бывший максимум становится вторым */
            max = a[i];
        } else if (a[i] > second && a[i] < max) {
            second = a[i]; /* не максимум, но больше второго */
        }
    }

    if (second == INT_MIN) {
        printf("Второго по величине нет.\n");
    } else {
        printf("Максимум: %d, второй: %d\n", max, second);
    }
    return 0;
}
```

Запуск:

```
Максимум: 9, второй: 7
```

Разбор. `INT_MIN` из `<limits.h>` — самое маленькое значение `int`. Стартуем с него, чтобы любой реальный элемент оказался больше. Массив `{3, 9, 9, 1, 7, 9, 4}` содержит максимум 9 трижды, но вторым выводится 7, а не 9 — условие `a[i] < max` отсекает дубликаты максимума. Это именно тот случай, который любят подсунуть.

Что спросят дальше.

- «Что если в массиве все элементы одинаковы (`{5,5,5}`)?» Тогда второго по величине нет — наш код напечатает «Второго нет», потому что `second` так и остался `INT_MIN`. Уточни у интервьюера, считать ли дубликат вторым: трактовки бывают разные, и важно проговорить выбор.
- «А k-й по величине?» Обобщение — там уже нужна частичная сортировка или куча (heap). Это уже уровень Middle.

Задача 12. Проверка на простое число

Условие. Дано число `n`. Определи, простое ли оно (делится только на 1 и на себя; 0, 1 и отрицательные — не простые).

Идея. Перебираем возможные делители от 2. Нашли хоть один — число составное. Ключевая оптимизация: проверять делители только **до корня** из `n`, потому что делители идут парами (`n = d · (n/d)`), и меньший из пары всегда не больше √n.

Решение (`code/19-07-prime.c`):

```c
#include <stdio.h>

/* Возвращает 1, если n — простое число, иначе 0. */
int is_prime(int n)
{
    if (n < 2) {
        return 0;             /* 0, 1 и отрицательные — не простые */
    }
    for (int d = 2; d * d <= n; d++) {
        if (n % d == 0) {
            return 0;         /* нашли делитель — составное */
        }
    }
    return 1;
```

```c
}

int main(void)
{
    int n;
    printf("Введи число: ");
    if (scanf("%d", &n) != 1) {
        printf("Это не число.\n");
        return 1;
    }

    if (is_prime(n)) {
        printf("%d — простое\n", n);
    } else {
        printf("%d — не простое\n", n);
    }
    return 0;
}
```

Запуск:

```
Введи число: 17
17 — простое
```

Разбор. Условие цикла `d * d <= n` вместо `d <= n` — это и есть «до корня»: вместо `sqrt()` сравниваем квадрат делителя с числом, без дробной арифметики. Для n = 1 000 000 это ~1000 проверок вместо миллиона. Случаи `n < 2` отсекаем сразу — иначе единица ошибочно сошла бы за простую.

Что спросят дальше.

- «Почему `d * d <= n`, а не `d <= sqrt(n)`?» Чтобы не звать `sqrt` (дробная функция, возможны погрешности на границе) и не подключать `<math.h>`. Целочисленное сравнение точнее и быстрее. Осторожно: для очень больших `n` `d * d` само может переполниться — тогда берут `long long` или сравнивают `d <= n / d`.
- «Ускорь ещё.» Проверять 2 отдельно, а дальше только нечётные делители (`d += 2`) — вдвое меньше итераций.
- «Все простые до N?» Это уже **решето Эратосфена** (см. главу 5) — частый следующий шаг.

Часть 2. Каверзы: «что выведет / в чём ошибка»

Здесь короткие фрагменты. На собесе их показывают и спрашивают: «что напечатает?» или «найди ошибку». Цель — не код написать, а проверить, насколько хорошо ты знаешь язык в деталях. Сначала подумай сам, потом читай ответ.

Каверза 1. `sizeof('a')` — сколько это?

```c
printf("%zu %zu\n", sizeof(int), sizeof('a'));
```

Вопрос. Что выведет? Подсказка-ловушка: `'a'` — это же один символ, значит 1 байт?

Ответ. Выведет `4 4` (на типичной системе). Сюрприз в том, что в языке **C** символьный литерал `'a'` имеет тип не `char`, а **`int`**! Поэтому `sizeof('a')` равен `sizeof(int)`, то есть 4, а не 1.

Важно: это особенность именно C. В C++ `'a'` — это `char`, и там `sizeof('a')` равно 1. На собесе по C правильный ответ — 4.

А вот `sizeof(char)` по стандарту **всегда** равен 1 — это и есть тот самый «единица измерения» для `sizeof`.

`%zu` — спецификатор для типа `size_t`, который возвращает `sizeof` (беззнаковый размер). Печатать его через `%d` формально неверно.

Каверза 2. Целочисленное деление `5 / 2`

```c
printf("%d\n", 5 / 2);
double x = 5 / 2;
printf("%f\n", x);
```

Вопрос. Что в обеих строках?

Ответ. Первая печатает `2` — деление двух **целых** идёт нацело, дробная часть отбрасывается (см. главу 2), не округляется. `5 / 2` это 2, а не 2.5 и не 3.

Вторая печатает `2.000000`, а **не** 2.5, и это главный подвох. Хотя `x` объявлен как `double`, выражение `5 / 2` считается **до** записи в `x`: оба операнда целые → целочисленное деление → 2 → и уже 2 превращается в `2.0`. Тип переменной слева не влияет на то, как вычисляется правая часть.

Чтобы получить 2.5, дробным должен быть хотя бы один операнд: `5.0 / 2`, или `(double)5 / 2`, или `5 / 2.0`.

Каверза 3. `printf("%d", c)` для `char`

```c
char c = 'A';
printf("%d\n", c);
```

Вопрос. Что выведет?

Ответ. Выведет `65`. Символ в C — это маленькое целое, его **код**. Код буквы `A` в таблице ASCII равен 65. Спецификатор `%d` просит напечатать число, поэтому печатается не сама буква, а её код. Если хочешь увидеть букву `A` — используй `%c`.

И наоборот полезно помнить: `printf("%c\n", 66)` напечатает `B` — число истолкуется как код символа. На этой двойственности (символ = число) держатся трюки вроде перевода регистра из задачи 8.

Каверза 4. `=` вместо `==` в условии

```c
int n = 5;
if (n = 0) {
    printf("ноль\n");
} else {
    printf("не ноль\n");
}
```

Вопрос. Что выведет?

Ответ. Выведет `не ноль`, и при этом **испортит** `n`. Здесь стоит один `=` (присваивание), а не `==` (сравнение). Поэтому `n = 0` сначала **кладёт** 0 в `n`, а потом результатом всего выражения становится само присвоенное значение — 0. Ноль в условии — это «ложь», поэтому срабатывает ветка `else`. Заодно `n` теперь равно 0, хотя мы вроде бы только «проверяли».

Если бы было `if (n = 5)`, условие стало бы истинным (5 — ненулевое), и такие баги особенно коварны: код «как будто работает». Спасает компилятор с `-Wall` — он предупреждает о присваивании внутри `if`. Защитный приём — «йода-условие»: писать `if (0 == n)`; тогда опечатку `0 = n` компилятор отвергнет как ошибку (нельзя присвоить константе).

Каверза 5. Лишняя `;` после `for(...)`

```c
int sum = 0;
for (int i = 1; i <= 5; i++);
{
    sum += i;
}
printf("%d\n", sum);
```

Вопрос. Найди ошибку. Что не так?

Ответ. Лишняя точка с запятой сразу после `for(...)`. Эта `;` — **пустое тело цикла**. То есть цикл крутится 5 раз и ничего не делает. А блок в фигурных скобках ниже — это уже **обычный блок**, не связанный с циклом; он выполнится **один раз**.

Хуже того: этот фрагмент вообще не скомпилируется — `i` объявлена внутри `for` и за его пределами не видна, так что `sum += i;` обратится к неизвестному имени. Но даже если бы `i` была видна, логика была бы сломана. Мораль: точка с запятой сразу после `for`, `while` или `if` — почти всегда ошибка. Компилятор с `-Wall` предупреждает об «empty body».

Каверза 6. `printf("%d %d", i++, ++i)`

```c
int i = 1;
printf("%d %d\n", i++, ++i);
```

Вопрос. Что выведет?

Ответ. Честный ответ: **неизвестно**. Это **неопределённое поведение** (undefined behavior, UB). В одном выражении переменная `i` и читается, и дважды меняется (`i++` и `++i`) без гарантированного порядка между этими изменениями. Стандарт C не определяет, в каком порядке вычисляются аргументы `printf` и когда применяются `++`. Разные компиляторы выдадут разные числа, и **полагаться нельзя ни на одно из них**.

Если на собесе спрашивают «что выведет» — правильный ответ не «2 3» и не «1 3», а именно: «это UB, результат не определён, так писать нельзя». Интервьюер проверяет, узнаёшь ли ты опасную конструкцию, а не считаешь ли в уме «как у меня на ноутбуке».

Лечится разнесением по строкам:

```c
int a = i++;   /* теперь порядок однозначен */
int b = ++i;
printf("%d %d\n", a, b);
```

Каверза 7. Неинициализированная переменная

```c
int sum;
for (int i = 1; i <= 5; i++) {
    sum += i;
}
printf("%d\n", sum);
```

Вопрос. Что выведет?

Ответ. Заранее **неизвестно**. `sum` объявлена, но не инициализирована — в локальной переменной лежит **мусор** (что осталось в этой ячейке памяти). К мусору прибавляется 1+2+3+4+5 = 15, и печатается «мусор + 15». Может вывестись и 15 (если повезло и там был 0), и любое другое число, и значение может меняться от запуска к запуску.

Чтение неинициализированной локальной переменной — тоже неопределённое поведение. Фикс — один символ: `int sum = 0;`. Любой накопитель перед циклом обнуляй (сумма — в 0, произведение — в 1). Компилятор с `-Wall` часто предупреждает «may be used uninitialized» — не игнорируй.

Каверза 8. Остаток от отрицательного числа

```c
printf("%d %d\n", -7 / 2, -7 % 2);
```

Вопрос. Что выведет?

Ответ. Выведет `-3 -1`. В современном C (стандарт C99 и новее) деление округляется **к нулю**: `-7 / 2` это −3 (а не −4). А остаток подчиняется правилу `(a / b) * b + a % b == a`, поэтому `-7 % 2` равен −1, а не +1.

Практический вывод: проверять нечётность через `n % 2 == 1` **опасно** — для отрицательных нечётных получится −1, и проверка провалится. Пиши `n % 2 != 0`. Это прямая связь с задачей 2 про чётность.

Мини-резюме

Что стоит унести из этого приложения — не сами решения (их в интернете тысячи), а привычки, которые отличают подготовленного кандидата:

- **Краевые случаи проговаривай вслух**: пустой массив, один элемент, ноль, отрицательное, дубликаты, переполнение. Интервьюер часто ставит плюс именно за то, что ты сам про них вспомнил.
- **Помни про типы и переполнение.** `int` лопается на ~2 млрд; символ — это `int` в C; деление целых идёт нацело. Эти три факта спрашивают чаще всего.
- **Знай про опасные приёмы, но не используй их без нужды.** XOR-своп, арифметический обмен — рассказать можешь, в код кладёшь `temp`.
- **UB называй своим именем.** Увидел `i++` и `++i` в одном выражении, неинициализированную переменную, изменение строкового литерала — отвечай «это неопределённое поведение», а не угадывай результат.
- **Компилируй с `-Wall -Wextra`.** Половину каверз из второй части компилятор ловит сам.

Все рабочие решения собираются и запускаются:

```
cc -std=c11 -Wall -Wextra -O2 code/19-07-prime.c -o prime
./prime
```

Куда дальше

Это был уровень Junior — задачи, где главное показать, что основы языка сидят крепко, а голова замечает краевые случаи. В **Приложении Б (Middle)** мы поднимем планку: связные списки и их развороты, работа с указателями всерьёз, своя реализация `qsort`-подобной сортировки и функций сравнения, хеш-таблицы, аккуратная работа с памятью под присмотром санитайзеров, и каверзы потяжелее — про указатели, выравнивание и время жизни объектов. Если задачи отсюда даются легко и без подглядывания — ты готов двигаться дальше.

Приложение Б. Задачи с собеседований: Middle

В Приложении А мы размялись на простых задачах уровня Junior. Теперь — ступень выше. Это вопросы, которые задают на собеседовании, когда хотят проверить, что ты понимаешь Си **по-настоящему**, а не просто умеешь печатать `printf`. Здесь всё крутится вокруг четырёх тем, на которых ломается большинство:

- **указатели** — кто на что смотрит и кто кого меняет;
- **память** — где живёт строка, когда `malloc`, когда `free`;
- **строки** — массив `char` с `'\0'` на конце (см. главу 3);
- **биты** — числа как набор нулей и единиц;
- **связные списки** — узлы и стрелки `next` (см. главу 11).

Как пользоваться этим приложением

Не читай решение сразу. Прочитай условие, закрой книгу и попробуй сам — хоть на бумаге, хоть в редакторе. На собеседовании ценят не зазубренный код, а ход мысли. Поэтому у каждой задачи есть блок «идея» (как рассуждать) и «доп. вопросы» — то, что интервьюер спросит следом, чтобы понять, не списал ли ты.

Задачи идут от простого к сложному. Сначала — **задачи на код** (их решения лежат отдельными файлами в `code/`, каждое собирается без предупреждений командой `cc -std=c11 -Wall -Wextra -O2`). Потом — **каверзы**: коварные вопросы про язык, где правильный ответ не «что выведет», а «почему так, и можно ли на это полагаться».

Поехали.

Часть 1. Задачи на код

Задача 1. Своя `strlen`, `strcpy`, `strcmp`

Условие. Реализуй три функции стандартной библиотеки `<string.h>` своими руками, без её вызова: длину строки, копирование, сравнение.

Идея. Строка в Си — это массив `char`, заканчивающийся нулевым символом `'\0'` (см. главу 3). Значит, любой обход строки — это «иди по символам, пока не упрёшься в `'\0'`». На этом держатся все три функции.

Решение (`code/20-01-mystr.c`):

```c
#include <stdio.h>

/* Своя длина строки: считаем символы до '\0'. */
size_t my_strlen(const char *s)
{
    size_t n = 0;
    while (s[n] != '\0') {  /* пока не дошли до конца строки */
        n++;
    }
    return n;
}

/* Своё копирование: льём src в dst посимвольно, вместе с '\0'. */
char *my_strcpy(char *dst, const char *src)
{
    char *start = dst;          /* запомнили начало, чтобы вернуть */
    while ((*dst = *src) != '\0') {
        dst++;
        src++;
    }
    return start;
}

/* Своё сравнение: 0 если равны, иначе разница первых отличий. */
int my_strcmp(const char *a, const char *b)
{
    while (*a != '\0' && *a == *b) {
        a++;
```

```
        b++;
    }
    return (unsigned char)*a - (unsigned char)*b;
}
```

Объяснение.

- `size_t` — это специальный беззнаковый тип для размеров и длин. Именно его возвращает настоящая `strlen`. Печатают `size_t` через `%zu`.
- `const char *s` — указатель на строку, которую мы **только читаем**. Слово `const` обещает «я не буду менять то, на что смотрит `s`». Это хороший тон: компилятор подстрахует, и тому, кто вызывает функцию, видно, что аргумент в безопасности.
- В `my_strcpy` трюк в строке `while ((*dst = *src) != '\0')`. Сначала `*dst = *src` копирует один символ **и** возвращает его значение. Это значение тут же сравниваем с `'\0'`. Получается: скопировали символ; если это был конец строки — выходим (но `'\0'` уже скопирован, что нам и нужно). Скобки вокруг `*dst = *src` обязательны: без них `!=` сработает раньше `=`.
- В `my_strcmp` идём, пока символы совпадают и строка не кончилась. На первом различии (или на общем конце) выходим и возвращаем разность кодов символов. Если строки равны, оба `*a` и `*b` — это `'\0'`, разность 0. Приведение к `unsigned char` нужно, чтобы символы с кодом больше 127 не «ушли в минус» (на некоторых системах `char` знаковый) — стандартная `strcmp` сравнивает именно как `unsigned char`.

Запуск:

```
my_strlen("Hello") = 5
my_strcpy -> skopiruem
my_strcmp(abc, abc) = 0
my_strcmp(abc, abd) = -1
my_strcmp(abd, abc) = 1
```

Доп. вопросы интервьюера.

- *Чем опасна `strcpy`?* Она не знает размер приёмника `dst` и пишет, пока в `src` не кончатся символы. Если `dst` короче — затрёт чужую память (переполнение буфера). Безопаснее `strncpy` или `snprintf`.
- *Почему `strcmp` возвращает не просто 1/-1, а разность?* Стандарт гарантирует только знак результата (меньше/равно/больше нуля), не само число. Полагайся на знак, не на конкретную величину.

Задача 2. Развернуть односвязный список

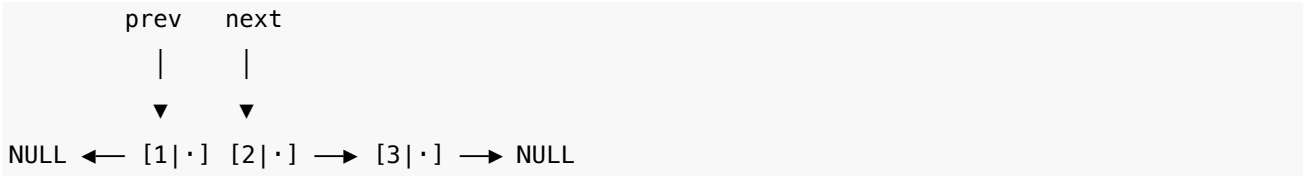
Условие. Дан односвязный список `1 -> 2 -> 3 -> ... -> NULL`. Разверни его, чтобы стало `... -> 3 -> 2 -> 1 -> NULL`. На месте, без выделения нового списка.

Идея. У каждого узла есть стрелка `next`, смотрящая вперёд. Развернуть список — значит у каждого узла **перенаправить стрелку назад**. Идём по списку и на каждом шаге разворачиваем одну стрелку. Но если просто написать `cur->next = prev`, мы потеряем доступ к хвосту — поэтому хвост надо запомнить **до** разворота. Нужны три указателя: `prev` (узел позади), `cur` (текущий), `next` (запасённый хвост).

Нарисуем. Было:



Запомнили `next = cur->next` (это `[2]`), развернули стрелку `cur` на `prev`:



Сдвинули `prev` и `cur` на шаг вперёд (`prev = cur; cur = next`) и повторяем. Когда `cur` станет `NULL`, `prev` будет указывать на бывший последний узел — это новая голова.

Решение (`code/20-02-reverse-list.c`, привожу ядро):

```c
struct Node {
    int value;
    struct Node *next;
};

/* Разворот списка через три указателя: prev, cur, next. */
struct Node *reverse(struct Node *head)
{
    struct Node *prev = NULL;       /* позади ничего нет */
    struct Node *cur = head;        /* бегунок по списку */
    while (cur != NULL) {
        struct Node *next = cur->next;  /* запомнили хвост */
        cur->next = prev;               /* развернули стрелку назад */
```

```
        prev = cur;                     /* prev сдвинулся вперёд */
        cur = next;                     /* cur сдвинулся вперёд */
    }
    return prev;                    /* prev — новая голова */
}
```

Объяснение. Порядок четырёх строк внутри цикла — священен.

1. `next = cur->next` — сохранили, куда идти дальше, пока стрелка цела.
2. `cur->next = prev` — развернули стрелку текущего узла назад.
3. `prev = cur` — `prev` догнал `cur`.
4. `cur = next` — `cur` шагнул на сохранённый хвост.

Поменяешь местами хоть две — либо потеряешь хвост, либо зациклишь узел сам на себя. Функция возвращает `prev`, потому что после цикла `cur` уже `NULL`, а `prev` стоит на последнем обработанном узле — он и есть новая голова. В `main` обязательно перезаписываем голову: `head = reverse(head)`.

Запуск:

```
До:    5 -> 4 -> 3 -> 2 -> 1 -> NULL
После: 1 -> 2 -> 3 -> 4 -> 5 -> NULL
```

Доп. вопросы.

- *Сложность?* Один проход — O(n) по времени, O(1) по памяти (три указателя, новый список не выделяем).
- *Как развернуть рекурсивно?* Можно (см. главу 9 про рекурсию), но тогда расход памяти O(n) на стек вызовов, и на длинном списке возможно переполнение стека. Итеративная версия лучше.

Задача 3. Середина списка (быстрый и медленный указатель)

Условие. Найди средний узел односвязного списка за один проход, не зная заранее его длину.

Идея. Самый прямой способ — пройти список дважды: сначала посчитать длину, потом дойти до середины. Но можно за **один** проход трюком «два указателя разной скорости». Медленный `slow` шагает по одному узлу, быстрый `fast` — по два. Когда `fast` добежит до конца, `slow` будет ровно посередине: он прошёл вдвое меньше.

Решение (`code/20-03-floyd.c`, ядро):

```
/* Середина списка: медленный шагает на 1, быстрый на 2. */
struct Node *middle(struct Node *head)
```

```
{
    struct Node *slow = head;
    struct Node *fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;          /* +1 */
        fast = fast->next->next;    /* +2 */
    }
    return slow;
}
```

Объяснение. Главное здесь — условие цикла `fast != NULL && fast->next != NULL`. Быстрый прыгает через два узла, и перед прыжком надо убедиться, что **оба** узла существуют: и сам `fast`, и `fast->next`. Иначе `fast->next->next` разыменует `NULL` — падение. Порядок проверок важен: `&&` вычисляет левую часть первой, и если `fast` уже `NULL`, до `fast->next` дело не дойдёт (это называется ленивым, или укороченным, вычислением — см. главу про условия). Для списка из 7 узлов середина — четвёртый, значение 4.

Запуск (первая строка):

```
Середина списка из 7: 4
```

Доп. вопросы.

- *А если узлов чётное число?* Тогда `slow` остановится на «правом из двух центральных». Хочешь левый — начни `fast` с `head->next` или останавливайся, когда `fast->next->next == NULL`.
- *Где ещё нужен этот приём?* В обнаружении цикла — следующая задача.

Задача 4. Цикл в списке (черепаха и заяц, алгоритм Флойда)

Условие. Список может быть «закольцован»: последний узел указывает не на `NULL`, а на один из предыдущих. Обычный обход по такому списку зациклится навечно. Определи, есть ли в списке цикл.

Идея. Та же пара указателей. Заяц (`fast`) бежит вдвое быстрее черепахи (`slow`). Если список заканчивается `NULL` — заяц добежит до конца, цикла нет. Если есть кольцо — заяц никогда не выйдет, он начнёт бегать по кругу и рано или поздно **догонит** черепаху сзади. Как на круговой беговой дорожке: быстрый бегун обязательно настигнет медленного. Момент `slow == fast` и означает цикл.

Решение (`code/20-03-floyd.c`, ядро):

```c
/* Обнаружение цикла: черепаха и заяц (Флойд). 1 если цикл есть. */
int has_cycle(struct Node *head)
{
    struct Node *slow = head;
    struct Node *fast = head;
    while (fast != NULL && fast->next != NULL) {
        slow = slow->next;
        fast = fast->next->next;
        if (slow == fast) {         /* заяц догнал черепаху */
            return 1;
        }
    }
    return 0;                       /* быстрый упёрся в NULL — цикла нет */
}
```

Объяснение. Сравниваем именно **указатели** (`slow == fast`), то есть адреса узлов, а не их значения: важно, что оба бегунка попали в один и тот же узел. Если цикла нет, `fast` выйдет на `NULL` и цикл `while` завершится. Почему заяц гарантированно догонит черепаху, а не «перепрыгнет»? Потому что за каждый шаг расстояние между ними внутри кольца сокращается ровно на 1, значит, оно непременно станет 0.

В программе мы сначала проверяем нормальный список (цикла нет), потом вручную закольцовываем его (последний узел направляем на третий) и проверяем снова:

```
Цикл есть? 0
После закольцовки цикл есть? 1
```

Важная деталь из кода: перед `free` цикл надо **разорвать** (`last->next = NULL`), иначе освобождение зациклится и мы дважды освободим одни и те же узлы.

Доп. вопросы.

- *Сложность?* O(n) по времени, O(1) по памяти. Альтернатива — хранить посещённые узлы в хеш-таблице: тоже O(n) времени, но O(n) памяти.
- *Как найти, с какого узла начинается цикл?* Это вторая часть алгоритма Флойда: после встречи поставить один указатель в голову и двигать оба по одному шагу — встретятся как раз в начале кольца.

Задача 5. Подсчёт единичных битов (popcount)

Условие. Дано целое число. Посчитай, сколько единиц в его двоичной записи. Например, 7 это `111` — три единицы; 8 это `1000` — одна.

Идея. Наивно: смотрим младший бит (`n & 1`), прибавляем его к счётчику, сдвигаем число вправо (`n >> 1`), повторяем, пока число не обнулится. Это делает столько шагов, сколько всего битов в числе (до 32). Красивее — **приём Брайана Кернигана**: операция `n &= n - 1` за один раз **гасит самый младший единичный бит**. Тогда число итераций равно числу единиц, а не общему числу битов.

Почему `n & (n - 1)` убирает младшую единицу? Вычитание единицы переворачивает младший единичный бит в 0, а все нули правее него — в единицы. Например, `n = 1100`, `n - 1 = 1011`. Их `&` даёт `1000` — младшая единица исчезла, остальное цело.

Решение (`code/20-04-popcount.c`, ядро):

```c
/* Наивный popcount: проверяем младший бит и сдвигаем вправо. */
int popcount_naive(unsigned int n)
{
    int count = 0;
    while (n != 0) {
        count += n & 1u;    /* младший бит: 0 или 1 */
        n >>= 1;            /* сдвинули число на один бит вправо */
    }
    return count;
}

/* Приём Брайана Кернигана: n &= n - 1 гасит младший единичный бит. */
int popcount_kernighan(unsigned int n)
{
    int count = 0;
    while (n != 0) {
        n &= n - 1;         /* убрали одну единицу */
        count++;            /* итераций ровно столько, сколько единиц */
    }
    return count;
}
```

Объяснение.

- Аргумент — `unsigned int` (беззнаковое). Битовые операции и сдвиги делают **только** на беззнаковых типах. Сдвиг вправо отрицательного знакового числа ведёт себя зависимо от компилятора — этого избегают.
- `n & 1u` — побитовое И с единицей оставляет только младший бит: результат 0 или 1. Суффикс `u` у литерала `1u` подчёркивает, что работаем с беззнаковым.

- `n >>= 1` — то же, что `n = n >> 1`: сдвиг всех битов на одну позицию вправо, как деление на 2.
- `n &= n - 1` — то же, что `n = n & (n - 1)`: гасим младшую единицу.

Запуск:

```
    0: бит=0 (Кернига=0), степень_двойки=0
    1: бит=1 (Кернига=1), степень_двойки=1
    2: бит=1 (Кернига=1), степень_двойки=1
    7: бит=3 (Кернига=3), степень_двойки=0
    8: бит=1 (Кернига=1), степень_двойки=1
  255: бит=8 (Кернига=8), степень_двойки=0
 1024: бит=1 (Кернига=1), степень_двойки=1
```

Обе функции дают одно число, но версия Кернигана для числа `1024` (это `10000000000`) сделает **один** виток цикла, а наивная — одиннадцать.

Доп. вопросы.

- *Есть ли это в железе?* Да, у многих процессоров есть инструкция popcount; в GCC/Clang к ней даёт доступ `__builtin_popcount`.
- *Как посчитать единицы в `unsigned long long`?* Тот же код, тип шире; следи, чтобы литералы и тип счётчика были достаточны.

Задача 6. Степень ли двойки

Условие. Проверь, является ли положительное число степенью двойки (1, 2, 4, 8, 16, …).

Идея. У степени двойки в двоичной записи **ровно один** единичный бит: `1`, `10`, `100`, `1000`. А мы только что видели, что `n & (n - 1)` гасит младшую единицу. Если единица была одна-единственная — после гашения останется 0. Значит, проверка: `n != 0 && (n & (n - 1)) == 0`.

Решение (`code/20-04-popcount.c`, ядро):

```c
/* Степень двойки: ровно один единичный бит, и число положительное. */
int is_power_of_two(unsigned int n)
{
    return n != 0 && (n & (n - 1)) == 0;
}
```

Объяснение. Условие `n != 0` обязательно. Без него ноль прошёл бы проверку: `0 & (0 - 1)` для беззнакового — это `0 & 0xFFFFFFFF`, что равно 0, и `is_power_of_two(0)` ошибочно вернуло

бы «истина». Ноль не степень двойки, поэтому отсекаем его явно. Результат — это значение логического выражения: 1 (истина) или 0 (ложь).

Доп. вопросы.

- *Почему не считать popcount и не сравнивать с 1?* Можно, но это лишний цикл. Трюк с `n & (n-1)` — за одну операцию, без ветвлений и циклов.
- *Как округлить до ближайшей степени двойки вверх?* Серия сдвигов и `|`, либо инструкция подсчёта ведущих нулей (`__builtin_clz`). Это уже уровень повыше.

Задача 7. Удалить дубликаты из отсортированного массива на месте

Условие. Дан отсортированный массив, например `[1, 1, 2, 2, 2, 3, 5, 5, 8]`. Убери повторы **на месте**, не выделяя второй массив. Верни новую длину; уникальные элементы должны оказаться в начале.

Идея. Раз массив отсортирован, все одинаковые элементы лежат рядом. Заведём два индекса (приём «два указателя» в одном массиве): `r` (reader) бежит по всем элементам, `w` (writer) показывает, куда положить следующий уникальный. Если `a[r]` отличается от последнего записанного `a[w-1]` — это новый уникальный, копируем его в `a[w]` и двигаем `w`.

Решение (`code/20-07-dedup.c`, ядро):

```c
/* Удаление дубликатов из ОТСОРТИРОВАННОГО массива на месте.
   Возвращает новую длину; уникальные элементы — в начале a. */
int dedup_sorted(int *a, int n)
{
    if (n == 0) {
        return 0;
    }
    int w = 1;                  /* w — куда писать следующий уникальный */
    for (int r = 1; r < n; r++) {   /* r — что читаем */
        if (a[r] != a[w - 1]) {     /* отличается от последнего записанного */
            a[w] = a[r];
            w++;
        }
    }
    return w;
}
```

Объяснение. Первый элемент `a[0]` уникален всегда, поэтому стартуем с `w = 1` и `r = 1`. Сравниваем именно с `a[w - 1]` — последним, что мы уже оставили, а не с соседом `a[r - 1]`:

так корректно пропускаем длинные серии повторов. Проверка `n == 0` нужна, иначе `a[w - 1]` при пустом массиве полез бы в `a[-1]` — выход за границу.

Запуск:

```
Уникальных: 5
1 2 3 5 8
```

Доп. вопросы.

- *Почему важно, что массив отсортирован?* Только тогда дубликаты рядом и хватает сравнения с предыдущим уникальным. Для неотсортированного понадобится множество (хеш) — O(n) памяти.
- *Что в хвосте массива после удаления?* Старый мусор за позицией `w`. Мы его не трогаем — вызывающий пользуется только первыми `w` элементами.

Задача 8. Число, встречающееся нечётное число раз (XOR)

Условие. В массиве все числа встречаются **парами**, и только одно — нечётное число раз. Найди его. Желательно за один проход и без лишней памяти.

Идея. Спасает операция XOR (исключающее ИЛИ, `^`). У неё два золотых свойства: `x ^ x == 0` (число, сложенное само с собой, гасится) и `x ^ 0 == x` (ноль ничего не меняет). Проксорим **все** элементы подряд. Парные погасят друг друга и дадут 0, а одиночка останется — это и есть ответ. Порядок неважен, XOR коммутативен.

Решение (`code/20-08-oddtimes.c`, ядро):

```c
/* XOR всех элементов: парные гасят друг друга, остаётся одиночка. */
int find_odd(const int *a, int n)
{
    int x = 0;
    for (int i = 0; i < n; i++) {
        x ^= a[i];      /* a ^ a == 0, x ^ 0 == x */
    }
    return x;
}
```

Объяснение. `x ^= a[i]` — то же, что `x = x ^ a[i]`. Стартуем с `x = 0`, потому что 0 — нейтральный для XOR. После прохода все, кто шёл парами, взаимно уничтожились, в `x` лежит число с непарным количеством. Память — одна переменная, время — один проход.

Запуск (4 встречается 3 раза):

```
Нечётное число раз встречается: 4
```

Доп. вопросы.

- *А если два числа непарные?* Тогда общий XOR даст их «смесь». Их можно разделить, выбрав бит, в котором они различаются, и разбив массив на две группы — это классическое усложнение задачи.
- *Где XOR ещё полезен?* Обмен двух чисел без временной переменной, простое шифрование, контрольные суммы.

Задача 9. Корректный бинарный поиск

Условие. Реализуй бинарный поиск в отсортированном массиве. Главная ловушка — ошибки «на единицу» (off-by-one): неверная граница цикла или неверный сдвиг `lo`/`hi` ведут к зацикливанию или пропуску элемента.

Идея. Держим отрезок `[lo, hi]`, где может находиться искомое. Смотрим середину. Угадали — готово. Если в середине меньше искомого — всё левее бесполезно, двигаем `lo`. Если больше — двигаем `hi`. Отрезок сжимается, пока не найдём или пока он не опустеет.

Решение (`code/20-05-binsearch.c`, ядро):

```c
/* Бинарный поиск в отсортированном массиве.
   Возвращает индекс target или -1, если не найдено. */
int binary_search(const int *a, int n, int target)
{
    int lo = 0;
    int hi = n - 1;             /* оба конца ВКЛЮЧИТЕЛЬНО */
    while (lo <= hi) {          /* пока отрезок непустой */
        int mid = lo + (hi - lo) / 2;   /* середина без переполнения */
        if (a[mid] == target) {
            return mid;
        } else if (a[mid] < target) {
            lo = mid + 1;       /* искомое правее */
        } else {
            hi = mid - 1;       /* искомое левее */
        }
    }
    return -1;
}
```

Объяснение. Три места, где обычно ошибаются, и как мы их обходим:

- **Границы включительные.** `hi = n - 1`, и цикл идёт, пока `lo <= hi` (не <). Когда `lo == hi`, отрезок ещё содержит один элемент — его нужно проверить. Напишешь `lo < hi` — пропустишь этот последний элемент.
- **Сдвиг с +1/–1.** После проверки середины сам `mid` уже исключён, поэтому новые границы — `mid + 1` и `mid - 1`, а не `mid`. Если сдвигать на `mid`, отрезок может перестать сжиматься и цикл зависнет.
- **Середина без переполнения.** Пишем `lo + (hi - lo) / 2`, а не `(lo + hi) / 2`. На больших индексах сумма `lo + hi` может переполнить `int`; разность `hi - lo` безопасна. Классический баг, который годами жил даже в стандартных библиотеках.

Запуск:

```
  5 найдено на индексе 4
 42 найдено на индексе 8
 -7 найдено на индексе 0
100 не найдено
  0 найдено на индексе 2
```

Доп. вопросы.

- *Сложность?* O(log n): каждый шаг вдвое урезает отрезок. Для миллиона элементов — около 20 шагов.
- *Что вернуть, если в массиве есть дубликаты искомого?* Базовая версия вернёт «какой-то» из них. Чтобы найти первый или последний, цикл меняют так, чтобы он не останавливался на совпадении, а продолжал сужать сторону.

Задача 10. Слияние двух отсортированных массивов

Условие. Даны два отсортированных массива. Слей их в один отсортированный. Это сердце сортировки слиянием (merge sort).

Идея. Идём по обоим массивам сразу, двумя индексами. Сравниваем текущие элементы и в результат кладём **меньший**, продвигая его индекс. Так — пока один из массивов не кончится. Остаток второго дописываем как есть: он уже отсортирован.

Решение (`code/20-06-merge.c`, ядро):

```c
/* Слияние двух отсортированных массивов a и b в результат out.
   out должен вмещать na + nb элементов. */
void merge(const int *a, int na, const int *b, int nb, int *out)
{
    int i = 0, j = 0, k = 0;
    while (i < na && j < nb) {       /* пока есть в обоих */
```

```
        if (a[i] <= b[j]) {
            out[k++] = a[i++];       /* меньший идёт первым */
        } else {
            out[k++] = b[j++];
        }
    }
    while (i < na) {                 /* хвост a, если остался */
        out[k++] = a[i++];
    }
    while (j < nb) {                 /* хвост b, если остался */
        out[k++] = b[j++];
    }
}
```

Объяснение. Запись `out[k++] = a[i++]` делает две вещи: кладёт `a[i]` в `out[k]`, а потом увеличивает оба индекса (постфиксный `++` сначала даёт старое значение, потом увеличивает — разбор `++` будет в каверзах ниже). Сравнение `a[i] <= b[j]` именно «меньше или равно» — это делает слияние **устойчивым** (равные элементы сохраняют исходный относительный порядок; для чисел незаметно, для записей с полями важно). Два хвостовых цикла дописывают то, что осталось от длинного массива.

Ответственность за размер `out` — на вызывающем: там должно быть место под `na + nb` элементов. В `main` массив `out[11]` именно такой.

Запуск:

```
1 2 3 4 5 6 7 8 9 10 12
```

Доп. вопросы.

- *Сложность?* O(na + nb) по времени — один проход по каждому. Память — под результат, O(na + nb).
- *Как слить «на месте» в конец первого массива (как в задачах про merge sorted array)?* Идти **с конца** обоих массивов, заполняя `out` справа налево — тогда не затрёшь ещё не прочитанные элементы.

Задача 11. Поменять местами два числа через указатели

Условие. Напиши функцию `swap`, которая меняет местами значения двух переменных. Так, чтобы изменения были видны **снаружи** функции.

Идея. В Си аргументы передаются по значению — функция получает копии (см. главу 3). Поменяв копии, оригиналы не тронешь. Чтобы дотянуться до самих переменных, передаём их **адреса** и работаем через указатели.

Решение (показываю целиком, это короткая классика):

```c
#include <stdio.h>

void swap(int *a, int *b)   /* принимаем адреса двух переменных */
{
    int t = *a;             /* запомнили значение по a */
    *a = *b;                /* в a положили значение по b */
    *b = t;                 /* в b положили запомненное */
}

int main(void)
{
    int x = 5, y = 9;
    swap(&x, &y);           /* передаём АДРЕСА x и y */
    printf("x=%d y=%d\n", x, y);   /* x=9 y=5 */
    return 0;
}
```

Объяснение. `int *a` — указатель: `a` хранит адрес чужой переменной. `*a` — значение по этому адресу, то есть сама переменная вызывающего. Меняя `*a` и `*b`, мы меняем `x` и `y` в `main`. При вызове пишем `&x` и `&y` — берём адреса. Именно поэтому `scanf` тоже хочет `&`: ему нужен адрес, чтобы записать в твою переменную.

Доп. вопросы.

- *Почему версия `void swap(int a, int b)` не работает?* Она меняет местами копии; снаружи всё остаётся как было.
- *Можно ли без временной переменной, через XOR?* Можно: `*a ^= *b; *b ^= *a; *a ^= *b;`. Но есть подвох: если `a` и `b` указывают на **одну** переменную, она обнулится. Версия с временной `t` надёжнее и читается лучше — на собеседовании предпочитают её.

Часть 2. Каверзы

Эти вопросы не про «реши задачу», а про «понимаешь ли ты язык». Любимая территория собеседующих: ответ короткий, но за ним стоит понимание памяти, указателей и неопределённого

поведения. Каждую каверзу разбираем, поэтому решений в `code/` для них нет — некоторые из них специально содержат ошибку, и компилировать их как «образец» нельзя.

Каверза 1. `char *s = "abc"` против `char s[] = "abc"`

```
char *s1 = "abc";       /* указатель на строковый ЛИТЕРАЛ */
char  s2[] = "abc";     /* МАССИВ-копия, лежит у тебя */

s2[0] = 'A';            /* ок: меняем свой массив, выйдет "Abc" */
s1[0] = 'A';            /* НЕЛЬЗЯ: запись в литерал — UB */
```

В чём разница. `s2` — это **массив**: компилятор выделил тебе личные 4 байта (`a`, `b`, `c`, `\0`) и скопировал туда содержимое литерала. Твой массив — меняй на здоровье.

`s1` — это **указатель** на строковый литерал `"abc"`. Литерал лежит в области памяти только для чтения. Попытка `s1[0] = 'A'` — это **неопределённое поведение** (undefined behavior, UB): чаще всего программа падает, но стандарт не обещает ничего. Полагаться на исход нельзя — может «сработать» на одной машине и упасть на другой.

Правило. Хочешь менять строку — заводи массив (`char s[]`). Указатель на литерал годится только для чтения; чтобы не забыть, объявляй его `const char *s1 = "abc";` — тогда компилятор сам запретит запись.

Каверза 2. `sizeof` массива внутри функции

```
void f(int arr[])               /* на самом деле int *arr ! */
{
    printf("%zu\n", sizeof arr);    /* размер УКАЗАТЕЛЯ: 8 (на 64-бит) */
}

int main(void)
{
    int a[10];
    printf("%zu\n", sizeof a);      /* размер МАССИВА: 40 (10 * 4) */
    f(a);
    return 0;
}
```

В чём подвох. В месте объявления `sizeof a` знает настоящий размер массива — 40 байт (10 чисел по 4). Но при передаче в функцию массив **распадается** (decays) в указатель на первый элемент. Параметр `int arr[]` — это синтаксическая маскировка под `int *arr`. Поэтому внутри

функции `sizeof arr` — это размер **указателя** (обычно 8 байт на 64-битной системе), а не массива.

Следствие. Внутри функции по самому массиву его длину не узнать. Поэтому длину **всегда передают отдельным параметром**: `f(int *arr, int n)`. Так устроены все наши функции в этой главе. Это же объясняет, почему `strlen` нужна строкам: длину массива `char` иначе не вычислить, спасает только метка `'\0'`.

Каверза 3. `arr[1]`, `1[arr]` и `*(arr + 1)`

```c
int arr[] = {10, 20, 30};
printf("%d\n", arr[1]);     /* 20 */
printf("%d\n", *(arr + 1)); /* 20 — то же самое */
printf("%d\n", 1[arr]);     /* 20 — и это тоже!? */
```

Объяснение. По определению языка `arr[i]` — это **ровно** `*(arr + i)`: «возьми адрес начала, прибавь `i`, разыменуй». Сложение коммутативно: `arr + i` и `i + arr` — одно и то же. Значит, `*(arr + i)` равно `*(i + arr)`, а это по тому же правилу записывается как `i[arr]`. Поэтому `1[arr]` — легальный (хоть и дикий) способ написать `arr[1]`.

`arr + 1` — это **арифметика указателей**: «следующий элемент», то есть адрес сдвигается не на 1 байт, а на размер одного элемента (`sizeof(int)`). Компилятор сам умножает на размер типа.

Зачем это знать. На собеседовании этим проверяют, понимаешь ли ты, что индексация — синтаксический сахар над арифметикой указателей. Писать `1[arr]` в реальном коде, разумеется, нельзя — это запутывает.

Каверза 4. `++*p`, `*p++`, `*++p`, `(*p)++`

Четыре похожих записи, которые делают **разное**. Разберём по очереди. Пусть:

```c
int a[] = {10, 20, 30};
int *p = a;             /* p смотрит на a[0] (значение 10) */
```

- **`++*p`** — это `++(*p)`. Разыменуй `p` (получим `a[0]`) и сделай префиксный инкремент значения. Результат: `a[0]` стало 11, `p` не двигался. Выражение даёт 11.
- **`(*p)++`** — постфиксный инкремент **значения**, на которое смотрит `p`. `a[0]` станет 11, но выражение вернёт **старое** значение 10. `p` на месте.
- **`*p++`** — здесь приоритет у постфиксного `++`: это `*(p++)`. Сначала берётся текущий `p` (адрес `a[0]`), потом `p` сдвигается на следующий элемент. Звёздочка разыменует **старый** адрес. Итог: выражение даёт 10 (значение `a[0]`), а `p` теперь смотрит на `a[1]`. Значения в массиве не менялись. Эта запись — рабочая лошадка при проходе по строке: `while (*p++)`.

- **`*++p`** — сначала префиксный `++p` (сдвигаем `p` на `a[1]`), потом разыменование. Выражение даёт 20, `p` смотрит на `a[1]`.

Как не путаться. Запомни приоритеты: постфиксный `++` цепляется к `p` сильнее, чем `*`. Поэтому `*p++` — это «разыменуй, а указатель сдвинь», а не «сдвинь значение». Если хочешь менять **значение** — ставь скобки: `(*p)++`. На собеседовании именно скобки показывают, что ты понимаешь, что делаешь.

Каверза 5. `const char *p` против `char * const p`

```
char a = 'a', b = 'b';

const char *p1 = &a;    /* нельзя менять *p1, можно менять p1 */
/* *p1 = 'x';  -- ошибка компиляции */
p1 = &b;                /* ок: указатель переставили */

char * const p2 = &a;   /* можно менять *p2, нельзя менять p2 */
*p2 = 'x';              /* ок: значение поменяли (a стало 'x') */
/* p2 = &b;  -- ошибка компиляции */
```

Как читать. Правило простое: читай объявление **справа налево от имени**.

- `const char *p1` → «`p1` — указатель на `const char`». Под запретом **то, на что смотрим** (`*p1`). Сам указатель свободно переставляется.
- `char * const p2` → «`p2` — `const`-указатель на `char`». Под запретом **сам указатель** (его нельзя перенаправить). А значение по нему менять можно.

Зачем. `const char *` — самая частая форма для параметров-строк, которые функция только читает (как в нашей `my_strlen`). А бывает и `const char * const p` — нельзя ни то, ни другое.

Каверза 6. Найди баг: утечка памяти

```
char *make_greeting(const char *name)
{
    char *buf = malloc(64);
    if (buf == NULL) return NULL;
    snprintf(buf, 64, "Привет, %s!", name);
    return buf;
}

int main(void)
```

```
{
    printf("%s\n", make_greeting("Аня"));   /* БАГ: утечка */
    return 0;
}
```

Где баг. `make_greeting` выделяет память через `malloc` и возвращает указатель. Но в `main` мы передаём этот указатель сразу в `printf` и **теряем** его — переменной, которая хранила бы адрес, нет. Освободить такой блок уже нечем: `free` вызвать не от чего. Это **утечка памяти**: блок занят до конца программы.

Как чинить. Сохрани адрес, попользуйся, освободи:

```
char *msg = make_greeting("Аня");
if (msg != NULL) {
    printf("%s\n", msg);
    free(msg);              /* на каждый malloc — свой free */
}
```

Правило из главы 3 в полный рост: на каждый `malloc` однажды должен прийтись ровно один `free`. Если функция **возвращает** выделенную память, в её документации/имени обычно намекают, что освобождать — обязанность вызывающего.

Каверза 7. Найди баг: use-after-free и двойной free

```
int *p = malloc(sizeof(int));
*p = 42;
free(p);                /* память возвращена системе */

printf("%d\n", *p);     /* БАГ 1: use-after-free — читаем мёртвое */
free(p);                /* БАГ 2: double free — освобождаем дважды */
```

Что не так. После `free(p)` блок больше не наш. Указатель `p` всё ещё хранит старый адрес (он стал «висячим», dangling), но трогать память по нему нельзя:

- `*p` после `free` — **use-after-free**: читаем (или пишем) освобождённую память. Неопределённое поведение: может показать старое значение, мусор или уронить программу.
- второй `free(p)` — **double free**: освобождение уже освобождённого блока. Тоже UB, частая причина падений и уязвимостей.

Как чинить. Не обращайся к памяти после `free`, а указатель сразу обнуляй:

```
free(p);
p = NULL;       /* теперь p честно «никуда»; free(NULL) безопасен */
```

Привычка `p = NULL` после `free` лечит оба бага разом: повторный `free(NULL)` по стандарту ничего не делает, а случайное `*p` по `NULL` упадёт сразу и заметно, а не испортит данные втихую.

Каверза 8. Возврат адреса локальной переменной

```
int *broken(void)
{
    int x = 42;
    return &x;          /* БАГ: x умрёт по выходе из функции */
}

int main(void)
{
    int *p = broken();
    printf("%d\n", *p); /* читаем мёртвую память — UB */
    return 0;
}
```

Где баг. `x` — локальная переменная, она живёт на стеке только пока работает `broken`. После `return` её память освобождается. Адрес `&x` становится висячим: данные по нему уже не наши. Обращение `*p` в `main` — неопределённое поведение. Иногда «случайно работает» (значение ещё не затёрто) — и это самое коварное, потому что баг прячется.

Как чинить. Либо вернуть **значение**, а не адрес (`int broken(void)` с `return x;`), либо выделить память динамически (`malloc`) и отдать ответственность за `free` вызывающему, либо принять буфер от вызывающего параметром. Возвращать адрес `static`-переменной можно, но это уже другая история (общая память на все вызовы).

Мини-резюме

Что должно остаться в голове после этого приложения:

- **Строки** — это `char`-массивы с `'\0'`; любая своя строковая функция идёт по символам до нуля. Литерал `"abc"` менять нельзя.
- **Списки** — три указателя разворачивают, два указателя разной скорости находят середину и ловят цикл (Флойд). Следи за порядком присваиваний и не разыменовывай `NULL`.

- **Биты** — `n & (n-1)` гасит младшую единицу (popcount Кернигана, проверка степени двойки); XOR гасит пары (одиночка за O(1) памяти).
- **Поиск и слияние** — бинарный поиск держи на включительных границах `[lo, hi]` с `lo + (hi - lo) / 2`; слияние — два индекса плюс хвосты.
- **Память** — на каждый `malloc` свой `free`; после `free` ставь `p = NULL`; не возвращай адрес локальной переменной; помни про decay массива в указатель и `const`-квалификаторы.

И главное, что проверяют на Middle: ты не зубришь алгоритмы, а понимаешь, **что происходит в памяти**. Поэтому на собеседовании проговаривай вслух, кто на что указывает и кто кого освобождает, — это ценится больше готового ответа.

Что дальше

Дальше — **Приложение В: Senior**. Там задачи злее: обобщённые структуры через `void *` и указатели на функции, своя мини-куча на `malloc`, выравнивание и упаковка `struct`, разбор хитрых объявлений вроде «указатель на функцию, возвращающую указатель на массив», устранение рекурсии, и каверзы про неопределённое поведение, переполнение знакового `int` и порядок вычисления аргументов. Если этот раздел дался без больших мучений — ты готов идти выше.

Приложение B. Задачи с собеседований: Senior

Это третий, самый трудный сборник задач с собеседований. Если в «Junior» проверяли, помнишь ли ты синтаксис, а в «Middle» — умеешь ли собрать алгоритм, то здесь смотрят глубже: понимаешь ли ты, **как Си устроен внутри**. Память, указатели на указатели, `void *`, выравнивание структур, неопределённое поведение (UB), тонкости препроцессора — то, на чём спотыкаются даже люди с опытом.

Задачи идут от простого к сложному и делятся на два блока:

- **на код** — написать рабочую программу: быстрые сортировки, обобщённые структуры через `void *`, хеш-таблица, кольцевой буфер, LRU-кэш и другие классические задачи;
- **каверзы и теория** — короткие вопросы, где надо не написать код, а объяснить, **почему так**: что выведет хитрый `sizeof`, чем опасен макрос `SQR(x)`, что такое strict aliasing.

Всё, что нужно, ты уже прошёл в главах 1–18: указатели и память (главы 3, 11), рекурсия (глава 9), сортировки и `qsort` (глава 8), структуры и связные списки (глава 11). Здесь мы доведём это до уровня, который ждут от senior-разработчика на Си.

Совет тот же, что и раньше: сначала реши задачу сам на бумаге и в редакторе, и только потом сверяйся с разбором. Все программы лежат в `code/21-*.c` и собираются без единого предупреждения:

```
cc -std=c11 -Wall -Wextra -O2 code/21-01-quicksort.c -o q && ./q
```

Часть 1. Задачи на код

Задача 1. Быстрая сортировка (quicksort)

Условие. Реализуй быструю сортировку массива `int` на месте (без дополнительного массива). Объясни выбор опорного элемента и сложность.

Идея. В главе 8 мы разобрали идею словами: выбираем **опорный** (pivot) элемент, переставляем массив так, чтобы слева оказались все меньшие опорного, а справа — большие. После такой перестановки опорный стоит на своём окончательном месте. Дальше рекурсивно сортируем левую и правую части.

Главная деталь — **разбиение** (partition). Возьмём схему Ломуто: опорным берём последний элемент `a[hi]`. Ведём границу `i` — индекс последнего «малого» элемента. Идём `j` от начала: как только встретили элемент меньше опорного, сдвигаем границу и меняем его туда. В конце ставим опорный сразу за границей.

Решение (`code/21-01-quicksort.c`):

```c
#include <stdio.h>

/* Обмен двух int через указатели (см. главу 3). */
void swap(int *a, int *b)
{
    int t = *a;
    *a = *b;
    *b = t;
}

/* Разбиение Ломуто: опорный — последний элемент a[hi]. */
int partition(int a[], int lo, int hi)
{
    int pivot = a[hi];          /* опорный (pivot) */
```

```
    int i = lo - 1;             /* граница «меньших опорного» */

    for (int j = lo; j < hi; j++) {
        if (a[j] < pivot) {     /* нашли элемент меньше опорного */
            i++;
            swap(&a[i], &a[j]);
        }
    }
    swap(&a[i + 1], &a[hi]);    /* опорный встаёт на своё место */
    return i + 1;
}

void quicksort(int a[], int lo, int hi)
{
    if (lo >= hi) {             /* 0 или 1 элемент — уже отсортировано */
        return;
    }
    int p = partition(a, lo, hi);
    quicksort(a, lo, p - 1);
    quicksort(a, p + 1, hi);
}
```

В `main` зовём `quicksort(a, 0, n - 1)` — границы **включительно**, поэтому `hi = n - 1`, а не `n`.

Объяснение.

- `swap(int *a, int *b)` принимает **адреса** переменных и меняет их содержимое через разыменование `*a`, `*b` (см. главу 3). Передай мы числа по значению — обмен случился бы только внутри функции.
- `int i = lo - 1;` — граница начинается «до» отрезка: малых элементов пока нет. Инвариант разбиения: всё в `a[lo..i]` меньше опорного, всё в `a[i+1..j-1]` — больше либо равно.
- цикл `for (j = lo; j < hi; j++)` доходит только до `hi - 1`, сам опорный `a[hi]` не трогаем — его поставим в конце.
- `swap(&a[i + 1], &a[hi])` — после цикла `i+1` это первая «большая» ячейка; меняем её с опорным, и тот оказывается ровно между меньшими и большими. Возвращаем `i + 1` — его финальную позицию.

Выбор опорного и сложность. В среднем разбиение делит массив пополам, уровней рекурсии около `log n`, на каждом уровне суммарно `n` сравнений — итого **O(n log n)**. Но беда в **худшем случае**: если опорный каждый раз оказывается минимумом или максимумом (а

на уже отсортированном массиве взятие последнего элемента именно это и даёт!), части получаются 1 и n−1, глубина рекурсии вырастает до n, и сложность скатывается в **O(n²)** — плюс риск переполнить стек.

Лечат это выбором опорного поумнее:

- **случайный опорный** — `swap(&a[hi], &a[lo + rand() % (hi − lo + 1)])` перед разбиением; отсортированный вход больше не «убивает» сортировку;
- **медиана трёх** — берут медиану из `a[lo]`, `a[mid]`, `a[hi]`; почти бесплатно и хорошо защищает от худшего случая.

Память: quicksort работает на месте, дополнительной памяти — только под стек рекурсии (O(log n) при хорошем опорном).

Доп. вопросы.

- Чем схема Хоара (два бегунка навстречу) лучше Ломуто? (Меньше обменов.)
- Почему quicksort **нестабильна** (равные элементы могут поменять порядок)?
- Как сделать quicksort нерекурсивной, на явном стеке?

Задача 2. Сортировка слиянием (merge sort)

Условие. Реализуй сортировку слиянием. В чём её плюс перед quicksort?

Идея. Делим массив пополам, рекурсивно сортируем каждую половину, потом **сливаем** две отсортированные половины в одну. Слияние — главный шаг: идём двумя бегунками по головам половин и каждый раз забираем меньший. В отличие от quicksort, merge sort даёт **гарантированные** O(n log n) даже в худшем случае и **стабильна**, но требует доп. памяти под временный буфер.

Решение (`code/21-02-mergesort.c`):

```c
void merge(int a[], int tmp[], int lo, int mid, int hi)
{
    int i = lo;          /* бегунок по левой половине */
    int j = mid + 1;     /* бегунок по правой половине */
    int k = lo;          /* куда пишем в tmp */

    while (i <= mid && j <= hi) {
        if (a[i] <= a[j]) {     /* <= даёт стабильность */
            tmp[k++] = a[i++];
        } else {
            tmp[k++] = a[j++];
```

```
        }
    }
    while (i <= mid) { tmp[k++] = a[i++]; }   /* хвост левой */
    while (j <= hi)  { tmp[k++] = a[j++]; }   /* хвост правой */
    for (int t = lo; t <= hi; t++) {          /* копируем обратно */
        a[t] = tmp[t];
    }
}

void merge_sort(int a[], int tmp[], int lo, int hi)
{
    if (lo >= hi) {
        return;
    }
    int mid = lo + (hi - lo) / 2;   /* без переполнения lo+hi */
    merge_sort(a, tmp, lo, mid);
    merge_sort(a, tmp, mid + 1, hi);
    merge(a, tmp, lo, mid, hi);
}
```

Объяснение.

- `tmp` — **один** временный буфер на весь алгоритм, выделяем его в `main` через `malloc((size_t)n * sizeof(int))` и передаём внутрь. Так не выделяем память на каждом слиянии.
- `int mid = lo + (hi - lo) / 2;` — почему не `(lo + hi) / 2`? Сумма `lo + hi` может **переполнить** `int` на огромных индексах. Запись `lo + (hi - lo)/2` даёт ту же середину, но без риска переполнения. Это классический приём (та же ловушка живёт и в бинарном поиске, см. главу 7).
- `a[i] <= a[j]` — именно `<=`, а не `<`: при равенстве берём элемент из **левой** половины, и равные сохраняют исходный порядок. Это и есть **стабильность**.
- три `while` подряд: первый сливает, пока живы обе половины; два следующих дописывают остаток той половины, что ещё не кончилась.

Quicksort или merge sort? Quicksort обычно быстрее на практике (лучше дружит с кэшем, сортирует на месте), но имеет худший случай $O(n^2)$. Merge sort всегда $O(n \log n)$ и стабилен, но тратит $O(n)$ доп. памяти. Для связных списков merge sort даже удобнее — слияние не требует случайного доступа.

Доп. вопросы.

- Сколько памяти под `tmp`? (O(n).) Можно ли слить две половины на месте за O(n)? (Сложно, обычными приёмами — нет.)
- Что такое «восходящий» (bottom-up) merge sort без рекурсии?

Задача 3. Обобщённый стек через `void *`

Условие. Напиши стек (или динамический массив), который умеет хранить элементы **любого** типа — `int`, `double`, структуру, — не переписывая код под каждый тип.

Идея. Шаблонов, как в C++, в Си нет. Универсальность строят на `void *` («указатель на что угодно») и на знании **размера элемента в байтах**. Стек хранит сырой байтовый буфер и `elem_size`. `push` **копирует** `elem_size` байт из переданного адреса в буфер, `pop` — обратно. Сам стек не знает, что за тип внутри: он просто двигает байты через `memcpy`.

Решение (`code/21-03-generic-stack.c`):

```c
typedef struct {
    char *data;          /* буфер из байтов */
    size_t elem_size;    /* размер одного элемента в байтах */
    size_t count;        /* сколько элементов лежит */
    size_t capacity;     /* сколько элементов влезает */
} Stack;

int stack_init(Stack *s, size_t elem_size)
{
    s->elem_size = elem_size;
    s->count = 0;
    s->capacity = 4;
    s->data = malloc(s->capacity * elem_size);
    return s->data != NULL;
}

int stack_push(Stack *s, const void *elem)
{
    if (s->count == s->capacity) {                /* места нет — растём */
        size_t new_cap = s->capacity * 2;
        char *bigger = realloc(s->data, new_cap * s->elem_size);
        if (bigger == NULL) {
            return 0;
        }
```

```
        s->data = bigger;
        s->capacity = new_cap;
    }
    memcpy(s->data + s->count * s->elem_size, elem, s->elem_size);
    s->count++;
    return 1;
}

int stack_pop(Stack *s, void *out)
{
    if (s->count == 0) {
        return 0;
    }
    s->count--;
    if (out != NULL) {
        memcpy(out, s->data + s->count * s->elem_size, s->elem_size);
    }
    return 1;
}
```

Использование одинаково для разных типов:

```
Stack si; stack_init(&si, sizeof(int));
int x = 10; stack_push(&si, &x);

Stack sd; stack_init(&sd, sizeof(double));
double d = 3.14; stack_push(&sd, &d);
```

Объяснение.

- буфер — `char *`, потому что `char` гарантированно 1 байт; адресная арифметика `data + count * elem_size` ходит по буферу побайтно и попадает ровно в начало нужного элемента. По `void *` так шагать **нельзя** — арифметика на `void *` стандартом не определена.
- `const void *elem` в `push` — «адрес элемента, тип неизвестен, менять не буду». Вызывающий передаёт `&x`. Внутри `memcpy` копирует `elem_size` байт — ровно один элемент.
- `pop` пишет результат по адресу `out` (если он не `NULL`) — тем же `memcpy`. Так мы «возвращаем» значение неизвестного типа: через выходной указатель, а не через `return`.
- `realloc` с удвоением вместимости — амортизированная вставка за O(1) (см. растущий массив в главе 11). Результат кладём во **временный** `bigger` и проверяем на `NULL`, иначе при неудаче потеряем старый блок.

Плата за обобщённость. Нет проверки типов: положил `int`, достал как `double` — компилятор смолчит, а ты получишь мусор. И `memcpy` чуть медленнее прямого присваивания. Зато один код работает со всем.

Доп. вопросы.

- Почему буфер `char *`, а не `void *`? (Арифметика указателей.)
- Как сделать обобщённую функцию печати? (Передать ещё указатель на функцию `void print(const void *)` — см. задачу 5.)
- Чем такой стек отличается от `qsort`, которая тоже принимает `void *` и размер элемента?

Задача 4. Свои `atoi` и `itoa` с обработкой краёв

Условие. Напиши `my_atoi` (строка → `int`) и `my_itoa` (`int` → строка). Корректно обработай знак, пробелы, мусор и **переполнение**, включая `INT_MIN`.

Идея. В `atoi` копим число пошагово: `result = result*10 + цифра`. Опасность — переполнение `int`. Поэтому копим в `long` (он шире) и после каждой цифры проверяем выход за `INT_MAX`/`INT_MIN`. В `itoa` главная каверза — `INT_MIN`: его модуль `-INT_MIN` в `int` **не помещается** (диапазон несимметричен: от −2147483648 до 2147483647). Спасает работа в `long`.

Решение (`code/21-04-atoi-itoa.c`):

```c
#include <limits.h>

int my_atoi(const char *s, int *out)
{
    while (*s == ' ' || *s == '\t') {   /* ведущие пробелы */
        s++;
    }

    int sign = 1;
    if (*s == '+' || *s == '-') {
        if (*s == '-') sign = -1;
        s++;
    }

    if (*s < '0' || *s > '9') {          /* нет цифр — не число */
        return 0;
    }
```

```
    long result = 0;                    /* копим в long: он шире int */
    while (*s >= '0' && *s <= '9') {
        result = result * 10 + (*s - '0');
        if (sign == 1 && result > INT_MAX) return 0;
        if (sign == -1 && -result < INT_MIN) return 0;
        s++;
    }

    *out = (int)(sign * result);
    return 1;
}

void my_itoa(int n, char *buf)        /* buf >= 12 байт */
{
    char tmp[12];
    int i = 0;
    int neg = 0;

    long value = n;                   /* long, чтобы взять -INT_MIN */
    if (value < 0) {
        neg = 1;
        value = -value;
    }

    do {                              /* цифры выходят задом наперёд */
        tmp[i++] = (char)('0' + value % 10);
        value /= 10;
    } while (value > 0);

    int j = 0;
    if (neg) buf[j++] = '-';
    while (i > 0) buf[j++] = tmp[--i];   /* разворачиваем в buf */
    buf[j] = '\0';
}
```

Вывод программы:

```
my_atoi("42") = 42
my_atoi("  -17") = -17
```

```
my_atoi("2147483648") = ошибка
my_atoi("-2147483648") = -2147483648
my_atoi("abc") = ошибка
my_itoa(-2147483648) = "-2147483648"
```

Объяснение.

- `*s - '0'` — перевод символа-цифры в число. Цифры в таблице ASCII идут подряд: `'7' - '0' == 7` (см. главу 10).
- проверки переполнения **после каждой цифры**: для плюса `result` превысил `INT_MAX`, для минуса `-result` ушёл ниже `INT_MIN`. `long` гарантированно вмещает любое промежуточное значение (на 64-битных системах он 8 байт).
- в `my_itoa` цикл `do { } while` обязателен: иначе для `n == 0` мы не выведем ни одной цифры. `do/while` гарантирует хотя бы один проход.
- `value = -value` для отрицательного — безопасно только потому, что `value` это `long`. Сделай мы `int value = n; value = -value;` при `n == INT_MIN` получили бы **UB** (переполнение знакового).
- `tmp[12]` хватает: максимум 10 цифр у `int` + знак + `'\0'` = 12.

Доп. вопросы.

- Почему `int abs(INT_MIN)` — это UB? (Результат не помещается в `int`.)
- Как `my_itoa` обобщить на любую систему счисления (`base`)?
- Что вернёт стандартный `atoi("99999999999")`? (UB при переполнении — поэтому в реальном коде берут `strtol`, который умеет сообщать об ошибке через `errno`.)

Задача 5. Таблица обработчиков на указателях на функции

Условие. Сделай диспетчер: по символу операции (+, −, ∗) вызывается нужная функция. Заодно покажи компаратор для `qsort`.

Идея. Функцию в Си можно передать как аргумент и хранить в переменной — через **указатель на функцию**. Связку «ключ → функция» складываем в массив структур (таблицу). Вместо длинного `switch` пробегаем таблицу и зовём найденный обработчик. Это тот же приём, что делает `qsort` универсальной: ей передают указатель на компаратор.

Решение (`code/21-05-funcptr.c`):

```c
int op_add(int a, int b) { return a + b; }
int op_sub(int a, int b) { return a - b; }
int op_mul(int a, int b) { return a * b; }
```

```c
typedef struct {
    char symbol;
    int (*fn)(int, int);   /* указатель на функцию (int, int) -> int */
} Handler;

int cmp_desc(const void *pa, const void *pb)
{
    int a = *(const int *)pa;
    int b = *(const int *)pb;
    return (a < b) - (a > b);   /* по убыванию, без вычитания a-b */
}

int main(void)
{
    Handler table[] = { {'+', op_add}, {'-', op_sub}, {'*', op_mul} };
    int n = (int)(sizeof table / sizeof table[0]);

    int x = 6, y = 4;
    char ops[] = {'+', '-', '*'};
    for (int k = 0; k < 3; k++) {
        for (int i = 0; i < n; i++) {
            if (table[i].symbol == ops[k]) {
                int r = table[i].fn(x, y);     /* вызов через указатель */
                printf("%d %c %d = %d\n", x, ops[k], y, r);
                break;
            }
        }
    }

    int a[] = {3, 1, 4, 1, 5, 9, 2, 6};
    int m = (int)(sizeof a / sizeof a[0]);
    qsort(a, (size_t)m, sizeof a[0], cmp_desc);
    /* ... печать: 9 6 5 4 3 2 1 1 */
    return 0;
}
```

Объяснение.

- `int (*fn)(int, int);` — самая пугающая запись в Си. Читай изнутри: `fn` — это указатель (`*fn`), на функцию, принимающую `(int, int)` и возвращающую `int`. **Скобки вокруг `*fn` обязательны**: без них `int *fn(int, int)` означало бы «функция, возвращающая `int *`» — совсем другое.
- имя функции `op_add` само по себе — уже её **адрес**, поэтому `{'+', op_add}` записывает указатель. Амперсанд не нужен (хотя `&op_add` тоже допустимо и означает то же).
- `table[i].fn(x, y)` — вызов через указатель. Можно писать и `(*table[i].fn)(x, y)` — это эквивалентно.
- `cmp_desc` возвращает `(a < b) - (a > b)`: даёт `+1`, `0` или `-1` без опасного `a - b` (которое может переполнить `int` на больших числах, см. ловушку в главе 8). Для убывания знак подобран так, что больший элемент идёт раньше.

Где это применяют по-настоящему. Таблицы команд в интерпретаторах, обработчики событий, колбэки (callbacks), state-машины, плагины. Указатель на функцию — основа любого «подключаемого» поведения в Си.

Доп. вопросы.

- Что такое `typedef int (*BinOp)(int, int);` и как он упрощает запись?
- Чем `qsort_r` (с доп. контекстом) удобнее обычной `qsort`?
- Можно ли сделать массив указателей на функции без структуры? (Да: `int (*ops[])(int,int) = {op_add, op_sub};`.)

Задача 6. Кольцевой буфер (ring buffer)

Условие. Реализуй кольцевой буфер фиксированного размера: запись в хвост, чтение из головы, переполнение и опустошение — корректно.

Идея. Это очередь (FIFO) поверх массива из главы 11, но оформленная аккуратной структурой. Храним массив, индекс головы `head` и количество `count`. Хвост вычисляем: `(head + count) % CAP`. Остаток от деления `%` заворачивает индекс с конца массива в начало — массив словно свёрнут в кольцо. Пуст при `count == 0`, полон при `count == CAP`.

Решение (`code/21-06-ringbuf.c`):

```c
#define CAP 4

typedef struct {
    int data[CAP];
    int head;     /* индекс первого элемента */
    int count;    /* сколько элементов лежит */
} Ring;
```

```c
int ring_push(Ring *r, int value)
{
    if (r->count == CAP) {
        return 0;                           /* буфер полон */
    }
    int tail = (r->head + r->count) % CAP;  /* первое свободное место */
    r->data[tail] = value;
    r->count++;
    return 1;
}

int ring_pop(Ring *r, int *out)
{
    if (r->count == 0) {
        return 0;                           /* буфер пуст */
    }
    *out = r->data[r->head];
    r->head = (r->head + 1) % CAP;          /* голова заворачивается */
    r->count--;
    return 1;
}
```

Объяснение.

- почему хранить `count`, а не индекс `tail`? Если держать `head` и `tail`, возникает путаница: при `head == tail` буфер пуст **или** полон — состояния неразличимы. С `count` всё однозначно.
- `(head + count) % CAP` — позиция за последним элементом, она же первое свободное место. Когда сумма дорастает до `CAP`, `%` сбрасывает её в 0.
- `head = (head + 1) % CAP` — после чтения голова шагает вперёд и тоже заворачивается.
- никаких `malloc` — буфер на статическом массиве внутри структуры, освобождать нечего.

В выводе видно, как после двух `pop` освободившиеся ячейки заняли `push 7` и `push 8`, завернувшись в начало:

```
push 5: полон
pop 1
pop 2
pop 7
pop 8
```

Зачем нужно. Кольцевые буферы — это потоки аудио/видео, буфер клавиатуры, логи фиксированного размера, обмен данными между потоком- производителем и потоком-потребителем. Размер заранее известен, память не скачет.

Доп. вопросы.

- Чем плох вариант с двумя индексами без `count`? (Неразличимы пусто/полно; лечат «потерей одной ячейки» или отдельным флагом.)
- Почему `CAP` — степень двойки иногда быстрее? (`% CAP` заменяется на `& (CAP-1)`.)
- Как сделать буфер растущим (динамическим)?

Задача 7. Хеш-таблица «строка → счётчик» с цепочками

Условие. Посчитай, сколько раз встретилось каждое слово. Реализуй хеш-таблицу с разрешением коллизий **цепочками**.

Идея. Хеш-таблица — массив «корзин». Функция-хеш превращает строку в число, остаток от деления на число корзин даёт индекс. Разные строки могут попасть в одну корзину — это **коллизия**. Решаем цепочками: каждая корзина это связный список записей, в ней лежащих. Поиск: вычислили корзину, прошли её цепочку, сравнили ключи `strcmp`.

Решение (`code/21-07-hashtable.c`):

```c
#define NBUCKETS 16

typedef struct Entry {
    char *key;
    int count;
    struct Entry *next;   /* цепочка внутри корзины */
} Entry;

typedef struct { Entry *buckets[NBUCKETS]; } HashTable;

unsigned long hash_str(const char *s)        /* djb2 */
{
    unsigned long h = 5381;
    int c;
    while ((c = (unsigned char)*s++) != 0) {
        h = h * 33u + (unsigned long)c;
    }
```

```
    return h;
}

int table_inc(HashTable *t, const char *key)
{
    size_t idx = hash_str(key) % NBUCKETS;

    for (Entry *e = t->buckets[idx]; e != NULL; e = e->next) {
        if (strcmp(e->key, key) == 0) {     /* слово уже есть */
            e->count++;
            return 1;
        }
    }

    Entry *e = malloc(sizeof(Entry));        /* новое слово */
    if (e == NULL) return 0;
    e->key = malloc(strlen(key) + 1);
    if (e->key == NULL) { free(e); return 0; }
    strcpy(e->key, key);
    e->count = 1;
    e->next = t->buckets[idx];               /* вешаем в начало цепочки */
    t->buckets[idx] = e;
    return 1;
}
```

Объяснение.

- `hash_str` (алгоритм djb2): начинаем с 5381, на каждый символ `h = h*33 + c`. Множитель 33 и старт 5381 подобраны эмпирически — такая функция хорошо «перемешивает» строки. Тип `unsigned long`, чтобы переполнение было определённым (для беззнаковых это не UB, а честный остаток по модулю — см. блок про UB ниже).
- `(unsigned char)*s` — важная деталь: `char` может быть знаковым, и русские/символы с кодом > 127 дали бы отрицательное число и кривой хеш. Приведение к `unsigned char` лечит это.
- коллизии: при `strcmp` мы **сравниваем сами ключи**, а не только хеши. Два разных слова с одинаковым хешем окажутся в одной корзине, но `strcmp` их различит.
- **владение памятью**: ключ копируем своим `malloc(strlen(key)+1) + strcpy`. Нельзя просто сохранить переданный указатель — строка вызывающего может измениться

или исчезнуть. На каждый `malloc` есть `free` в `table_free` (сначала `free(e->key)`, потом `free(e)`).

- вставка в **начало** цепочки — O(1), порядок внутри корзины неважен.

Про сложность. При хорошей хеш-функции и небольшом коэффициенте заполнения цепочки коротки, и поиск/вставка — в среднем **O(1)**. Если таблица переполнена (элементов много больше корзин), цепочки длинные, и всё вырождается в **O(n)**. Лечат **рехешированием**: при заполнении увеличивают число корзин и перераскладывают записи.

Доп. вопросы.

- Чем цепочки отличаются от **открытой адресации** (линейное/квадратичное пробирование)? Плюсы и минусы.
- Что такое коэффициент загрузки (load factor) и когда делать рехеш?
- Почему `NBUCKETS` берут простым числом или степенью двойки?

Задача 8. Точка пересечения двух односвязных списков

Условие. Даны два односвязных списка, которые в какой-то момент сливаются в общий хвост. Найди **узел слияния** (по адресу, не по значению). Если не пересекаются — `NULL`.

Идея. Наивно — для каждого узла A проверять весь B: O(n·m). Умнее: если списки сливаются, у них **общий хвост одинаковой длины**. Значит, разница только в длине «головных» частей. Посчитаем длины, проматаем длинный список вперёд на разницу — и теперь оба узла на равном расстоянии до конца. Шагаем параллельно: первый узел, где **адреса совпадут**, и есть точка пересечения. Это O(n + m) времени и O(1) памяти.

Решение (`code/21-08-list-intersection.c`):

```c
typedef struct Node {
    int value;
    struct Node *next;
} Node;

int length(Node *head)
{
    int len = 0;
    for (Node *p = head; p != NULL; p = p->next) len++;
    return len;
}

Node *intersection(Node *a, Node *b)
```

```
{
    int la = length(a);
    int lb = length(b);

    while (la > lb) { a = a->next; la--; }   /* выравниваем старты */
    while (lb > la) { b = b->next; lb--; }

    while (a != b) {        /* сравниваем АДРЕСА узлов, не значения */
        a = a->next;
        b = b->next;
    }
    return a;               /* общий узел или NULL */
}
```

Объяснение.

- ключевая мысль — сравнивать `a != b`, то есть **указатели**, а не `a->value != b->value`. Пересечение списков означает, что с какого-то узла они физически идут по **одной и той же** памяти. Значения могут совпадать и случайно, а адрес узла уникален.
- после выравнивания обоим до конца идти поровну. Если есть общий хвост, бегунки войдут в него синхронно и встретятся на узле слияния. Если пересечения нет, оба одновременно дойдут до `NULL` — и `a == b == NULL`, цикл закончится, вернём `NULL`. Один и тот же код покрывает оба случая.
- освобождение тут с подвохом: общий хвост нельзя `free` дважды. Сначала чистим уникальные части A и B **до** общего узла `tail`, потом — сам хвост один раз. Двойной `free` — это UB (см. блок ниже).

Доп. вопросы.

- Как найти **начало цикла** в списке (алгоритм Флойда «черепаха и заяц», два указателя с разной скоростью)?
- Как решить без подсчёта длин? (Два указателя: дойдя до конца своего списка, перепрыгивают на начало чужого; встретятся на пересечении.)
- Почему сравнение по значению дало бы неверный ответ?

Задача 9. Набросок LRU-кэша

Условие. Реализуй кэш на N записей с вытеснением **давно не использованных** (LRU — Least Recently Used). `get` и `put` — в среднем за O(1).

Идея. Нужны две вещи сразу: быстрый **поиск по ключу** и быстрое определение, **кто давно не использовался**. Поиск даёт хеш-таблица. Порядок использования — **двусвязный список**: голова это самый свежий, хвост — самый старый кандидат на вылет. Каждый узел живёт одновременно в обеих структурах. `get`/`put` находят узел через хеш и переставляют его в голову списка. При переполнении удаляем хвост — и из списка, и из хеша.

Ключевой код (`code/21-09-lru.c`):

```c
typedef struct Node {
    int key, value;
    struct Node *prev, *next;   /* список «по свежести» */
    struct Node *hnext;         /* цепочка хеш-корзины */
} Node;

int lru_get(LRU *c, int key, int *out)
{
    Node *n = hash_find(c, key);
    if (n == NULL) return 0;
    list_unlink(c, n);          /* вынули из текущего места */
    list_push_front(c, n);      /* подняли в голову — самый свежий */
    *out = n->value;
    return 1;
}

void lru_put(LRU *c, int key, int value)
{
    Node *n = hash_find(c, key);
    if (n != NULL) {            /* уже есть — обновим и освежим */
        n->value = value;
        list_unlink(c, n);
        list_push_front(c, n);
        return;
    }
    if (c->size == c->capacity) {   /* полон — вытесняем хвост (старейший) */
        Node *old = c->tail;
        list_unlink(c, old);
        /* убрать old из его хеш-цепочки */
        Node **pp = &c->buckets[bucket_of(old->key)];
        while (*pp != old) pp = &(*pp)->hnext;
```

```
        *pp = old->hnext;
        free(old);
        c->size--;
    }
    n = malloc(sizeof(Node));
    if (n == NULL) return;
    n->key = key; n->value = value;
    n->hnext = c->buckets[bucket_of(key)];
    c->buckets[bucket_of(key)] = n;
    list_push_front(c, n);
    c->size++;
}
```

Объяснение.

- узел в **двух** структурах: `prev/next` — порядок свежести, `hnext` — цепочка хеша. Это и даёт O(1): хеш находит узел мгновенно, а двусвязный список позволяет вынуть узел из середины за O(1) (у односвязного пришлось бы искать предыдущий).
- `list_unlink` + `list_push_front` — «освежить»: при каждом обращении узел переезжает в голову. Значит, у хвоста всегда самый давний доступ.
- интересна строка `Node **pp = &c->buckets[...]; while (*pp != old) ...` — **указатель на указатель**. Он позволяет удалить узел из односвязной хеш-цепочки без отдельной проверки «это голова или середина»: `pp` всегда указывает на то поле (`next` предыдущего или сам `buckets[b]`), которое надо перенаправить. Классический senior-приём.
- вывод подтверждает логику: после `get 1` ключ 1 стал свежим, и `put 3` вытеснил именно 2 (давно не трогали), а 1 уцелел:

```
get 1 = 100 (теперь 1 свежий)
get 2 = промах (вытеснен)
get 1 = 100
get 3 = 300
```

Доп. вопросы.

- Почему именно **дву**связный список, а не односвязный? (Удаление узла из середины за O(1).)
- Чем LRU отличается от LFU (вытеснение по частоте) и FIFO?
- Как тот же кэш сделать потокобезопасным? (Мьютекс вокруг операций.)

Задача 10. Разворот числа с защитой от переполнения

Условие. Развернуть цифры знакового `int` (`123 → 321`, `-456 → -654`). Если перевёрнутое число не помещается в `int` — сообщить об ошибке, а не выдать мусор.

Идея. Снимаем последнюю цифру `n % 10`, отбрасываем её `n /= 10`, дописываем в результат `result = result*10 + цифра`. Ловушка та же, что в `atoi`: `result*10` может переполнить `int`. Но здесь нельзя «копить в long и проверить потом» как универсальный приём собеседования — часто требуют остаться в `int`. Поэтому проверяем **до** умножения: не превысит ли `result*10 + цифра` границу.

Решение (`code/21-10-reverse-int.c`):

```c
#include <limits.h>

int reverse_int(int n, int *out)
{
    int result = 0;
    while (n != 0) {
        int digit = n % 10;   /* знак сохраняется: -123 % 10 == -3 (C99) */
        n /= 10;

        if (result > INT_MAX / 10 ||
            (result == INT_MAX / 10 && digit > INT_MAX % 10)) {
            return 0;         /* переполнение вверх */
        }
        if (result < INT_MIN / 10 ||
            (result == INT_MIN / 10 && digit < INT_MIN % 10)) {
            return 0;         /* переполнение вниз */
        }
        result = result * 10 + digit;
    }
    *out = result;
    return 1;
}
```

Объяснение.

- работаем **без** перевода в `unsigned` или `long` — прямо в `int`. Поэтому `result` хранит уже со знаком: для отрицательного `n` и `digit`, и `result` отрицательны (в C99 деление/остаток для отрицательных округляются **к нулю**: `-123 / 10 == -12`, `-123 % 10 == -3`).

- проверка «влезет ли `result*10 + digit`» сделана **арифметикой без переполнения**: сравниваем `result` с `INT_MAX/10`. Если строго больше — умножение уже переполнит. Если ровно равно — смотрим на последнюю цифру относительно `INT_MAX % 10`. Симметрично для `INT_MIN`. Так мы ни разу не выходим за границы `int` в самих вычислениях.
- `1534236469` развернётся в `9646324351` — это больше `INT_MAX` (2147483647), поэтому функция честно вернёт «переполнение», а не мусор.

Доп. вопросы.

- Почему опасно «развернуть, потом проверить»? (Переполнение знакового — это UB ещё **до** проверки.)
- Как развернуть строку на месте (два указателя навстречу)? (См. главу 10.)
- Чем отличается поведение `%` для отрицательных в C89 и C99?

Часть 2. Каверзы и теория

Здесь код не пишут — объясняют. Эти вопросы отделяют тех, кто «знает синтаксис», от тех, кто понимает Си. Отвечай по существу и с примером.

Каверза 1. Что такое UB и где оно прячется

Неопределённое поведение (undefined behavior, UB) — это ситуации, которые стандарт Си **не описывает вовсе**. Компилятор вправе делать что угодно: выдать ожидаемый результат, выдать мусор, уронить программу или (самое коварное) «оптимизировать» твой код, считая, что UB невозможно. Программа с UB не «иногда работает» — она **некорректна**, даже если сегодня выводит правильное.

Запомни главное правило: **на UB полагаться нельзя**. Поведение может меняться от компилятора, флагов оптимизации, версии и даже соседних строк кода. Частые источники:

```
/* 1. Переполнение знакового int — UB */
int x = INT_MAX;
x = x + 1;              /* НЕ «станет INT_MIN», а UB */

/* 2. Разыменование NULL */
int *p = NULL;
int y = *p;             /* падение или хуже */

/* 3. Выход за границы массива */
int a[3];
a[3] = 5;               /* записали мимо — UB */

/* 4. Неинициализированная память */
int z;
printf("%d", z);       /* читаем мусор — UB */
```

```c
/* 5. Изменение строкового литерала */
char *s = "hello";
s[0] = 'H';            /* литерал в read-only памяти — UB/падение */

/* 6. Использование памяти после free */
free(p);
*p = 7;                /* «use after free» — UB */
```

Почему переполнение **знакового** — это UB, а беззнакового — нет? Для `unsigned` стандарт прямо говорит: считаем по модулю 2^n (заворачивается). А для знакового представление формально не зафиксировано, поэтому переполнение оставили неопределённым — и компиляторы этим пользуются для оптимизаций (например, считают, что `x + 1 > x` всегда истинно).

Лечение: не выходить за границы, инициализировать переменные, проверять указатели на `NULL`, для изменяемых строк заводить `char s[] = "hello";` (массив, а не указатель на литерал), не трогать память после `free`. Под рукой держи санитайзеры: `cc -fsanitize=address,undefined` ловит многие UB на лету.

Каверза 2. Strict aliasing — почему `int *` и `float *` нельзя путать

Отдельный, неочевидный вид UB. Правило **strict aliasing** гласит: к одному и тому же участку памяти нельзя обращаться через указатели **несовместимых** типов. Компилятор считает, что `int *` и `float *` никогда не указывают на одно место, и оптимизирует, опираясь на это.

```c
float f = 1.0f;
int *p = (int *)&f;    /* смотрим на float как на int */
*p = 0;                /* нарушение strict aliasing — UB */
```

Так часто пытаются «достать биты» числа. Из-за strict aliasing результат непредсказуем при оптимизации. Правильный способ — через `memcpy` или `union`:

```c
int bits;
memcpy(&bits, &f, sizeof bits);   /* законно: memcpy работает с байтами */
```

Исключение из правила — тип `char *` (и `unsigned char *`): через него **можно** смотреть на байты чего угодно. Поэтому обобщённый стек из задачи 3 хранил буфер как `char *` — это безопасно.

Каверза 3. `volatile` — зачем он нужен

`volatile` говорит компилятору: «эта переменная может измениться **в любой момент** способом, которого ты не видишь, — не кэшируй её, не выкидывай обращения, читай каждый раз заново».

Где это критично:

```
volatile int *reg = (volatile int *)0x40000000;  /* аппаратный регистр */
while (*reg == 0) { }   /* ждём, пока железо изменит регистр */
```

Без `volatile` компилятор увидит, что в цикле `*reg` не меняется, и превратит `while (*reg == 0)` в вечный цикл, прочитав регистр один раз. `volatile` запрещает такую оптимизацию.

Второй классический случай — переменная, меняемая в обработчике сигнала:

```
volatile sig_atomic_t stop = 0;   /* флаг «пора остановиться» */
```

Чего `volatile` **не** делает: он **не** обеспечивает потокобезопасность и атомарность. Для многопоточности нужны `_Atomic` / мьютексы, а не `volatile`. Это частая путаница на собеседовании.

Каверза 4. Две роли `static`

Слово `static` в Си означает **разные вещи** в зависимости от места.

1. Внутри функции — переменная живёт от первого вызова до конца программы (а не создаётся заново каждый раз) и сохраняет значение между вызовами:

```
int next_id(void)
{
    static int id = 0;   /* создаётся один раз, помнит значение */
    return ++id;
}
/* вызовы вернут 1, 2, 3, ... */
```

Инициализатор `= 0` срабатывает **один раз**. Память — не на стеке, а в статической области.

2. На уровне файла (у глобальной переменной или функции) — `static` ограничивает видимость **этим файлом** (внутренняя компоновка). Из других `.c`-файлов к ней не достучаться:

```
static int counter;          /* виден только в этом файле */
static void helper(void) { } /* приватная функция модуля */
```

Это способ скрыть детали реализации — аналог `private`. Без `static` глобальные имена видны всей программе, и легко получить конфликт имён при сборке из нескольких файлов.

Каверза 5. Stack vs heap и причины segfault

В программе на Си память делится на области; новичку важны две:

- **Стек** (stack) — здесь живут локальные переменные и кадры вызовов функций. Выделяется и освобождается **автоматически**, быстро, но размер ограничен (обычно несколько МБ). Бесконечная рекурсия его переполняет — **stack overflow** (см. главу 9).
- **Куча** (heap) — память от `malloc`/`calloc`/`realloc`. Большая, живёт, пока сам не вызовешь `free`. За ней следишь **ты**: забыл `free` — утечка; освободил дважды — UB.

```
int arr[1000000];     /* на стеке — рискуешь переполнить стек */
int *arr = malloc(1000000 * sizeof(int));  /* на куче — надёжнее для больших */
```

Segmentation fault («сегфолт») — аварийное завершение при обращении к памяти, которая тебе не принадлежит. Типичные причины:

- разыменование `NULL` или «висячего» указателя (после `free`);
- выход далеко за границы массива;
- запись в строковый литерал;
- переполнение стека глубокой рекурсией.

Сегфолт — это ещё везение: программа хотя бы падает сразу. Хуже, когда UB тихо портит данные.

Каверза 6. Выравнивание и padding: что выведет `sizeof`

```
struct A {
    char  c;    /* 1 байт */
    int   i;    /* 4 байта */
    char  d;    /* 1 байт */
};
printf("%zu\n", sizeof(struct A));   /* НЕ 6, а обычно 12 */
```

Почему не `1 + 4 + 1 = 6`? Процессор читает данные эффективнее, когда они лежат по «круглым» адресам: `int` (4 байта) должен начинаться с адреса, кратного 4. Это **выравнивание** (alignment). Чтобы его соблюсти, компилятор вставляет невидимые **байты-заполнители** (padding):

```
[ c ][pad][pad][pad][  i (4)  ][ d ][pad][pad][pad]
  0    1    2    3    4 5 6 7     8   9   10  11
```

- после `c` (адрес 0) — 3 байта padding, чтобы `i` встал на адрес 4;
- `d` на адресе 8;
- в конце ещё 3 байта, чтобы **весь размер** структуры был кратен выравниванию самого «требовательного» поля (`int`, 4). Это нужно, чтобы в массиве структур каждая следующая тоже была выровнена.

Итого 12. А если переставить поля по убыванию размера — padding почти исчезнет:

```
struct B {
    int   i;    /* 4 */
    char  c;    /* 1 */
    char  d;    /* 1 */
};              /* sizeof == 8: i, c, d, и 2 байта хвоста */
```

Правило: **группируй поля от больших к меньшим** — структура займёт меньше. Точные числа зависят от платформы; узнать выравнивание типа можно через `_Alignof(int)`.

Каверза 7. Опасный макрос `#define SQR(x) x*x`

```
#define SQR(x) x*x
```

Препроцессор делает **текстовую подстановку**, не понимая арифметики. Отсюда два бага.

Баг 1 — приоритет операций. `SQR(a + b)` развернётся в `a + b*a + b`, то есть `a + (b*a) + b`, а вовсе не $(a+b)^2$:

```
int r = SQR(a + b);   /* a + b*a + b — НЕ то, что хотели */
```

Лечится скобками вокруг **каждого аргумента и всего выражения**:

```
#define SQR(x) ((x) * (x))
```

Теперь `SQR(a + b)` → `((a + b) * (a + b))` — верно.

Баг 2 — двойное вычисление аргумента. Даже с правильными скобками `SQR(x)` подставляет `x` **дважды**. Если аргумент имеет побочный эффект, он сработает два раза:

```
int i = 3;
int r = SQR(i++);     /* ((i++) * (i++)) — i увеличится ДВАЖДЫ, результат — мусор */
```

Это UB вдобавок (см. каверзу 8 про порядок). От этого скобки не спасают — лекарство только одно: **не передавай в макросы выражения с побочными эффектами**, а лучше используй `static inline`-функцию:

```
static inline int sqr(int x) { return x * x; }
```

Функция вычислит аргумент один раз и уважает типы. В современном Си макрос для такого уже почти не нужен.

Каверза 8. Идиома `do { ... } while(0)` в макросах

Как написать макрос из нескольких инструкций, чтобы он вёл себя как одна? Наивно:

```
#define SWAP(a, b) { int t = a; a = b; b = t; }
```

Сломается в `if`-без-скобок из-за лишней `;`:

```
if (x) SWAP(p, q); else foo();   /* ; после } разрывает if/else — ошибка */
```

Канонический приём — обернуть тело в `do { ... } while(0)`:

```
#define SWAP(a, b) do { int t = (a); (a) = (b); (b) = t; } while (0)
```

Теперь `SWAP(p, q);` — это один корректный оператор с точкой с запятой на конце, и он спокойно живёт в `if/else`. `while(0)` гарантирует ровно один проход, а компилятор с оптимизацией выкинет фиктивный цикл. Это стандартная идиома многострочных макросов в Си.

Каверза 9. Порядок вычисления аргументов не определён

```
int i = 1;
printf("%d %d\n", i++, i++);   /* что напечатается? */
```

Ответ: **неизвестно**. Стандарт не задаёт, в каком порядке вычисляются аргументы функции — слева направо, справа налево или вперемешку. Может выйти `1 2`, может `2 1`. Более того, изменять `i` дважды без точки следования между изменениями — это **UB**, а не просто «зависит от компилятора».

То же относится к выражениям вроде:

```
a[i] = i++;          /* UB: и читаем, и меняем i без упорядочивания */
foo(i++, i++);       /* UB */
x = f() + g();       /* порядок вызовов f и g не определён */
```

Если `f()` и `g()` имеют побочные эффекты и зависят от порядка — поведение непредсказуемо. Вывод: **не меняй одну переменную несколько раз в одном выражении** и не полагайся на порядок вычисления аргументов. Разнеси по разным строкам.

Каверза 10. Endianness — порядок байтов

Многобайтовое число (`int` — 4 байта) можно хранить в памяти двумя способами:

- **little-endian** — младший байт первым (x86, ARM по умолчанию, большинство ПК);
- **big-endian** — старший байт первым (некоторые сетевые протоколы, старые процессоры; «сетевой порядок» — big-endian).

Проверить порядок на своей машине можно так:

```
int x = 1;
unsigned char *p = (unsigned char *)&x;   /* смотрим на байты числа */
if (p[0] == 1) {
    printf("little-endian\n");   /* младший байт (1) лежит первым */
} else {
    printf("big-endian\n");
}
```

Число `1` в памяти это байты `01 00 00 00` (little) или `00 00 00 01` (big). Читаем первый байт через `unsigned char *` — это **разрешено** strict aliasing (см. каверзу 2). Если там `1` — машина little-endian.

Когда это важно: чтение бинарных файлов и сетевых пакетов, обмен данными между разными машинами. Поэтому в сети числа гоняют в фиксированном порядке через `htons`/`htonl` (host-to-network) и обратно `ntohs`/`ntohl`.

Итог практикума

Ты прошёл три сборника задач с собеседований — Junior, Middle и Senior — и это достойный финал всей книги. Пробежимся по тому, что осело.

На уровне **кода** ты теперь умеешь не просто «сортировать массив», а реализовать quick-sort и merge sort с пониманием их сложности, худших случаев и выбора опорного. Ты собрал **обобщённые** структуры через `void *` и размер элемента, написал хеш-таблицу с разрешением коллизий цепочками, кольцевой буфер, нашёл пересечение списков за O(n+m) и набросал LRU-кэш на связке «двусвязный список + хеш». Это уже не учебные игрушки, а кирпичики, из которых сложены настоящие программы.

На уровне **понимания языка** ты разобрался с тем, что отделяет крепкого Си-разработчика от новичка: что такое UB и почему на него нельзя полагаться, чем опасны strict aliasing и переполнение знакового, зачем нужны `volatile` и две роли `static`, как устроены стек и куча и откуда берётся segfault, почему `sizeof` хитрой структуры больше суммы полей (выравнивание и padding), как не подорваться на макросе `SQR(x)` и зачем макросам идиома `do { } while(0)`, и почему порядок вычисления аргументов и endianness — вещи, о которых нельзя забывать.

Напутствие

Если ты дочитал до этих строк и хотя бы половину задач решил сам — поздравляю, ты уже не новичок. Ты прошёл путь от первого `printf` (глава 1) до LRU-кэша и тонкостей UB. Си не прощает небрежности, и именно поэтому он так многому учит: каждый указатель, каждый `malloc`, каждый байт выравнивания заставляют **понимать**, а не угадывать.

Что делать дальше:

- **пиши код каждый день** — алгоритмы забываются без практики; бери задачи, реализуй, ломай, чини;
- **читай чужой Си** — исходники SQLite, ядра Linux, Redis; там живут лучшие образцы того, как пишут на этом языке;

- **гоняй санитайзеры и предупреждения** — `-Wall -Wextra`, `-fsanitize=address,undefined`, valgrind: пусть инструменты ловят UB за тебя;
- **не бойся низкого уровня** — теперь ты понимаешь, что под капотом, и любой язык повыше будет казаться проще.

Спасибо, что прошёл эту книгу до конца. Главного ты уже добился — научился думать как программист, который **знает, что происходит в памяти**. Остальное — дело практики. Удачи на собеседованиях и в реальных проектах.